Certified Shortcuts and Depth Bounds for Full Optimal Binary Search Trees

Research checkpoint: exact shortcuts and barriers to relaxations

OpenAI Codex Sydney

Problem selection and sponsorship: the repository owner

October 19, 2026

Abstract

We investigate exact optimal binary search trees with ordered weights on both keys and gaps. Our latest construction is a reduction to the published core-sparse Monge multiplication and witness algorithms. Starting from a large terminal penalty, successive partial-tree layers form decreasing Monge upper bounds. Each layer is an explicit penalty ramp plus a bounded residual. A dyadic argument bounds all intermediate cores, including the early layers before a full balanced tree is feasible. For nonnegative integer weights of total mass $W = O(n)$, the reduction gives exact $O(n^{3/2}\log^{5/2}(n+1))$ construction, including reconstruction and allowing zero weights. Dense audits and condensed Python/C++ constructors verify the reduction and full reconstruction. The implemented backend loses to matched Knuth construction at the tested sizes through 2048 keys; we report the traversal and witness costs behind that negative result.

An implemented portfolio provides complementary exact shortcuts: key and block contractions, minimum- or bounded-excess path-length classes, dominant-gap separators, and joint heavy-outcome cores. A coupled two-gap certificate retains the first violating path skeleton and bounds its payloads with mixed forests. We also implement the established short-interval/top-skeleton architecture with a variable mass contour. A directly proved mixed Fibonacci mass bound and a root-search cancellation argument give $O(n\log(n+1) + \min_d\{S_d + n2^d\})$ arithmetic work, where S_d counts admitted intervals. On the specified positive linear-mass families this yields exponent below 1.59024; the core-sparse reduction improves that exponent, up to logarithmic factors, in the integer-mass regime.

We distinguish compression from information loss. Two positive mixed intervals have identical responses to all nonnegative endpoint-gap weight additions but different optima in the same surrounding context. A separate example has quadratically many affine response regions yet a linear-size exact factorization. Depth-order relaxations and local repairs also have explicit obstructions. These are scoped representation barriers, not a general computational lower bound.

Exact enumeration, independent cubic recurrences, adversarial inputs, source-hashed data, and operation counts accompany the implementations. The unrestricted subquadratic question remains open. Mathematical priority and broad practical superiority remain unestablished; the measured compact backend regressions and existing algorithms used by the reductions are explicit.

1 Model, scope, and prior work

Keys have fixed inorder positions $0, \ldots, n-1$ and nonnegative weights $p_0, \ldots, p_{n-1}$. The $n+1$ gaps have weights $q_0, \ldots, q_n$. A key at depth d contributes $p(d+1)$; a gap at depth d contributes qd. Charging an extra comparison at each gap adds the tree-independent constant $\sum q_i$.

For a half-open key interval $[i, j)$, write

$$W(i,j) = \sum_{r=i}^{j-1} p_r + \sum_{r=i}^{j} q_r, \qquad C(i,i) = 0.$$

The exact recurrence is

$$C(i,j) = W(i,j) + \min_{i \le r < j} \{C(i,r) + C(r+1,j)\}.$$

Knuth's monotone-root optimization computes the table in $O(n^2)$ time [1]. The central obstacle here is the number of states, not merely the cost per state. We retain both kinds of weights throughout.

Larmore gives subquadratic exact construction under restrictions on long runs of very low frequency items, together with an approximation tradeoff [2]. Karpinski, Larmore, and Rytter combine exact small intervals with a bounded top-level construction [3]. These results are essential context: an improvement on a restricted family must not be described as the first subquadratic result. This checkpoint does not make a comprehensive priority claim. The conditions below were derived in this investigation; a targeted literature search did not establish whether equivalent rules already exist.

All running-time bounds below count arithmetic operations and comparisons. They are not bit-complexity bounds for unbounded integer weights.

2 A sharper single-key contraction

A key is a *cherry* when its two children are external nodes. Replacing a cherry with one gap is exact: if the key and its gaps have weights p, x, y, their total is $s = p + x + y$. A cherry rooted at depth d contributes $s(d+1)$, whereas the replacement gap contributes sd.

Theorem 1 (Single-key rule)**.** *Let a key of weight p have adjacent gap weights x, y. If its immediate predecessor and successor keys have weights a, b and*

$$\min(a, b) \;\ge\; 2p + x + y,$$

there is an optimal tree in which this key is a cherry. At the left endpoint, it suffices that the successor weight is at least $p + x$; at the right endpoint, it suffices that the predecessor weight is at least $p + y$. Consequently contraction to a gap of weight $s = p + x + y$ satisfies $\mathrm{OPT}(\textit{original}) = \mathrm{OPT}(\textit{contracted}) + s$.

Proof. Consider the key at depth d in any tree. If its left subtree is nonempty, the immediate predecessor has depth $d + l$, $l \ge 1$. If its right subtree is nonempty, the immediate successor has depth $d + r$, $r \ge 1$. Set the corresponding distance to zero for an empty internal side.

When $l \ge 1$, move the predecessor to the chosen key's position, replacing the predecessor's old position by its left child. Insert the chosen key as a cherry at the leftmost gap of the former

right subtree. Inorder order is preserved. All other outcomes either retain their depths or become shallower. The predecessor saves la, and the cost change is at most

$$-la + (r+1)p + (r-l+1)x + y. \tag{1}$$

The mirrored successor exchange has change at most

$$-rb + (l+1)p + x + (l-r+1)y. \tag{2}$$

If both sides are nonempty and $l \geq r$, substituting $a \geq 2p + x + y$ in (1) gives

$$(r+1-2l)(p+x) + (1-l)y \leq 0.$$

Use (2) when $r \geq l$. If only the left side is nonempty, $r = 0$ and $a \geq p + y$ makes (1) nonpositive. The other case is symmetric. The stated interior condition implies these one-sided conditions. At a global endpoint only the appropriate one-sided case is possible. With both sides empty the key is already a cherry.

Applying the exchange to an optimum proves the existence claim. Contraction subtracts exactly s. Conversely, every contracted tree expands to a valid original tree at additional cost s, proving equality. □

The weaker condition $\min(a, b) \geq p + x + y$ is false. For $p = (24, 14, 24)$ and $q = (1, 0, 9, 0)$, the middle key has local mass 23 and neighbors of weight 24. Contracting it forces cost 142, while the unrestricted optimum costs 130.

Maintain a doubly linked list of remaining keys and weighted gaps. Initially queue every key; after a contraction, only its predecessor and successor need rechecking. At most $3n$ queue entries are processed. Saving the contracted tree at the new gap permits reconstruction. Thus this pass takes $O(n)$ operations and space, and a Knuth solve on the remaining m keys gives total time $O(n + m^2)$. No confluence or minimum-core-size property is claimed.

3 Whole-block contraction

Let $B = [i, j)$ be a nonempty proper interval, including its endpoint gaps q_i, q_j. Put $W = W(i, j)$ and $C = C(i, j)$, with costs measured from a stand-alone root of depth zero.

Theorem 2 (Block rule)**.** *If both boundary keys exist and their weights satisfy*

$$p_{i-1} \geq W + C, \qquad p_j \geq W + C,$$

there is an optimum in which all of B forms one subtree. For a prefix or suffix block, it suffices that the sole boundary key has weight at least C. Contracting the block to one gap of weight W then gives

$$\mathrm{OPT}(\textit{original}) = \mathrm{OPT}(\textit{contracted}) + C.$$

Proof. In any tree, let v be the lowest common ancestor of gaps q_i, q_j, at depth d. Its key lies in B, and every outcome of B descends from v. Hence the original contribution of B is at least $(d+1)W$.

Within the subtree rooted at v, delete the keys of B from the inward fringes of its two children. On the left, a deleted key has no retained right child and can be suppressed in favor of its left child; on the right the process is symmetric. Let the retained outside trees be A, D. All their retained outcomes are at depths no greater than before. If A is nonempty, its maximum is boundary key $i - 1$; if D is nonempty, its minimum is boundary key j.

Suppose both are nonempty, and their boundary keys had original depths $d+l, d+r$. If $l \geq r$, remove the maximum key from A, replacing its position by its left child, and make that key the new root at v. Its left child is the remainder of A and its right child is D. Put an optimal tree for B at the leftmost external position of D. This position has depth at most $d+r+1$. No outside outcome other than the promoted boundary moves deeper, and that boundary saves lp_{i-1}. The increase in the contribution of B is at most

$$C + (d+r+1)W - (d+1)W = C + rW.$$

Since $l \geq r \geq 1$ and $p_{i-1} \geq W + C$, the saving is at least $l(W+C) \geq rW + C$. The mirrored construction applies for $r \geq l$.

If only one retained outside tree is nonempty, promote its inward boundary key to v and attach the optimal block as the other child. The block increase is at most C, while that boundary saves at least its own weight. The interior hypothesis suffices, and for a global prefix or suffix the weaker stated hypothesis suffices. If both outside trees are empty, B already occupies the subtree at v and can be optimized in place.

The construction never increases cost. Apply it to an optimum. A block at depth e contributes $C + eW$, while its replacement gap contributes eW. Contraction and expansion now prove the equality. □

A balanced k-key tree has height $h_k = \lceil \log_2(k+1) \rceil$ and cost at most $h_k W$. Thus the interior test

$$\min(p_{i-1}, p_j) \geq (1 + h_k)W$$

and endpoint test $p_{\text{boundary}} \geq h_k W$ certify a block without first solving it. These are sufficient conditions, not characterizations.

4 An adaptive linear selection pass

Build the maximum Cartesian tree of the key-weight sequence, breaking ties deterministically. Its subtree intervals form a laminar family: intervals are disjoint or nested. Compute their sizes and masses by one traversal and prefix sums. Test each proper interval with the balanced-tree bound above. This inspects only $O(n)$ candidates. The restriction to Cartesian intervals is an algorithmic choice, not a claim that all useful blocks occur there.

For a threshold b, select the maximal certified intervals whose sizes are at most b. Different selected intervals have a surviving separator key between them. Contracting one therefore leaves every other selected interval's weights and boundary-key weights unchanged, so the block theorem applies successively. Write their sizes as $k_1, \ldots, k_s$, and $m = n - \sum_t k_t$. The exact number of nonempty interval-DP states is

$$S(b) = \tau(m) + \sum_t \tau(k_t), \qquad \tau(u) = \frac{u(u+1)}{2}.$$

We choose a threshold minimizing $S(b)$ without running these DPs. For every certified interval I of size k, find its maximal proper certified descendants J. Immediately before I becomes eligible, these descendants are its selected blocks. Its activation therefore changes the core size and block-state total by

$$\Delta m = -\left(k - \sum_J |J|\right), \qquad \Delta S_{\text{blocks}} = \tau(k) - \sum_J \tau(|J|).$$

These sums are computed bottom-up. Bucket activation events by integer size $1, \ldots, n$ and scan the buckets, maintaining m and the block-state total. There are $O(n)$ events and buckets. Equal-size intervals are disjoint; evaluate a threshold after processing its entire bucket. The empty selection $b = 0$ is included.

Corollary 3. *The selected threshold and blocks are found in $O(n)$ arithmetic operations and space. Solving the blocks and surviving core by Knuth's method and expanding their saved trees takes*

$$O\left(n + m^2 + \sum_t k_t^2\right)$$

time and $O(n + m^2 + \max_t k_t^2)$ space when block tables are released after each solve. The chosen state count is no larger than $n(n+1)/2$. Worst-case time remains $O(n^2)$.

The minimization is only over these size thresholds in this candidate hierarchy, not over every possible partition. It does not minimize wall-clock time. Preprocessing, table allocation, root scans, and reconstruction still count as work.

For perspective, a certified partition with $m = O(n^{2/3})$ surviving keys and every $k_t = O(n^{1/3})$ gives $O(n^{4/3})$ time, since $\sum k_t^2 \le (\max k_t)n$. This is a conditional structural consequence, not an unrestricted bound.

5 Recursive construction with a bounded preprocessing depth

The implementation `recursive_contract.py` combines the proved components. At each call it applies the single-key pass, compares the order-free lower bound of Section 15 with a feasible bisection-tree cost, and returns immediately on equality. Otherwise it selects blocks as above. It recurses only if their combined table-state estimate is at most three quarters of the full estimate; small components and unsuccessful decompositions fall back to Knuth's method.

Suppose a component has $m > 8$ keys after the single-key pass and is successfully decomposed. Every child component of size s satisfies $s(s+1) \le \frac{3}{4}m(m+1)$, hence $s < 0.92m$. Child sizes sum to m. Consequently the recursion has logarithmic depth, and the total number of keys processed on any recursion level is at most n. Linear selection and reconstruction cost $O(n \log n)$ in total. The heap-based equality checks cost $O(n \log^2 n)$ over all levels. If the terminal DP sizes are r_t, the construction therefore takes

$$O\left(n \log^2 n + \sum_t r_t^2\right)$$

arithmetic operations. Its worst case is still quadratic. This threshold prevents repeated expensive scans after arbitrarily small reductions. Correctness follows by contraction, exact equality certificates, and induction on the strictly smaller recursive components; reconstruction restores the original key indices and gap subtrees.

The recursive implementation passed 3,000 further random inputs through $n = 30$ against the cubic oracle and direct tree evaluation. A separate nested fixture checks composition across multiple decomposition levels. Matched Python timings use five inputs per family, alternate which solver runs first, and include preprocessing and reconstruction for both solvers. At $n = 1024$, the median runtime ratio (recursive divided by Knuth) was 0.134 on the positive-weight heavy-separator family; its median terminal DP count was 30,017 versus 524,800. On random mixed weights the ratio was 0.986 and on near-uniform weights 1.001, which support no meaningful gain. At $n = 512$, the respective random and near-uniform ratios were 1.131 and 1.093: preprocessing can make the

method slower. These are exploratory measurements on one M4 host, not production-language or cross-machine claims.

6 A certified shortcut through nearly complete shapes

Let $I(T)$ be the sum of internal-node depths, with root depth zero. The external path length is $I(T) + 2n$, by induction on the two subtrees. Consequently uniform key weight a and gap weight b give cost

$$(a + b)I(T) + (a + 2b)n.$$

For an arbitrary instance set $a = \min_i p_i$, $b = \min_j q_j$, and let H' be the order-free mixed-node Huffman bound of Section 15 on residual weights $p_i - a, q_j - b$. These weights remain nonnegative. Write $I_*(n)$ for the minimum possible unweighted internal path length, and let C_* be the best weighted cost among all trees with $I(T) = I_*(n)$.

Theorem 4 (Minimum-path-length certificate)**.** *For $n \geq 1$, if*

$$C_* \;\leq\; (a + b)(I_*(n) + 1) + (a + 2b)n + H',$$

then a tree attaining C_* is a globally optimal full OBST.

Proof. Split each outcome weight into its uniform baseline and residual. Every tree outside the minimum-I class has integer $I(T) \geq I_*(n) + 1$, and its residual cost is at least H'. Its full cost is therefore at least the right-hand side. Within the class, C_* is minimal by definition. Equality is sufficient, including when $a + b = 0$. □

Optimize the class without interval tables. Put $h = \lfloor \log_2(n + 1) \rfloor$, $S = 2^h$, $m = S - 1$, and $r = n - m$. Every minimum-I tree has the complete upper skeleton with m keys, and expands exactly r of its S external slots into single-key cherries. Indeed, a missing shallower slot could otherwise receive a deepest internal leaf, strictly reducing I. Inorder relabeling preserves search-tree feasibility in this unweighted shape argument. Thus

$$I_*(n) = (h - 2)2^h + 2 + hr.$$

Process skeleton slots $j = 0, \ldots, S - 1$ in inorder. State (j, e) records the number of processed slots and expansions, so the next input index is $i = j + e$. An unexpanded slot charges hq_i; an expanded slot charges $(h+1)(p_i+q_i+q_{i+1})$. Except after the last slot, also charge the next skeleton key at comparison length

$$d_j = h - v_2(j + 1),$$

using p_i after an unexpanded slot and p_{i+1} after an expansion. Here v_2 counts trailing zero bits. Exactly r expansions are required, so state (j, e) is retained only when

$$\max(0, r - (S - j)) \leq e \leq \min(r, j).$$

With $s = \min(r, S - r)$, each stage has at most $s + 1$ states and each state has two outgoing choices. This computes C_* and backpointers in $O(n(1 + s))$ time and space. A linear Cartesian-tree pass converts the chosen comparison lengths into the labeled tree. No key or gap weights are discarded in these transitions.

There is an exact early rejection test. Let F be the cost obtained by assigning independently sorted key and gap weights to the shallowest depths of a minimum-I shape. This is feasible for

the order-free residual relaxation, so $H' \le F - [(a+b)I_* + (a+2b)n]$. Therefore $C_* > F + a + b$ proves the certificate cannot succeed. Otherwise the sorted lists are reused in a two-queue Huffman implementation: nonnegative weights and nondecreasing internal weights make the sequence of merged weights nondecreasing. This adds linear work after sorting. The queue optimization is standard and is not claimed as novel.

The implementation attempts this DP only when $s \le \lfloor \log_2 n \rfloor + 1$. Including the residual bound, preprocessing, and reconstruction, a successful attempt costs $O(n \log n)$ arithmetic operations and space. A skipped or failed attempt falls back to Knuth, or to the preceding recursive construction. Both versions remain exact and have quadratic worst-case time. The test is a certificate, not an assumption that near-uniform inputs have balanced optima. Novelty of this shortcut has not been established.

Verification and limits. Minimum-I optimization matched independent enumeration of all tree shapes through eight keys on 360 weighted inputs. Both fallback versions matched independent cubic DP and direct tree costs on 3,000 further mixed-weight inputs through thirty keys; the unrestricted certificate succeeded on 768. At $n = 1,024$, one seeded input with every weight in $\{999, 1000, 1001\}$ used 2,048 shape states, 3,070 transitions, 10,511 sorting comparisons, and 1,350 merge comparisons, certifying cost 19,479,590. The nonminimum-I lower bound was 19,480,123. Knuth used 524,800 states and 1,050,139 root candidates. These work units have different costs; the state ratio is not a timing claim.

The selected perturbation sweep certified amplitude 8 around 1,000 at $n = 256$, amplitude 4 at $n = 512$, and amplitude 1 at $n = 1,024$; larger tested amplitudes at those sizes failed. These are seeded observations, not universal thresholds. Ordinary random inputs failed the certificate. Sizes 384, 768, and 1,000 deliberately skip the shortcut because their minimum-I shape space exceeds the logarithmic width budget.

Six matched trials per fixture used all six orderings of Knuth, recursive contraction, and the combined solver, with identical inputs and correctness checks outside timing. On Apple M4 with Python 3.9.6, median per-trial combined/Knuth time ratios at $n = 1,024$ were 0.03934 for near-uniform, 0.10478 for heavy separators, 1.05454 for random, 0.91929 for log-uniform, and 1.03117 for gap spikes. All six near-uniform cases certified. The skipped near-uniform controls at $n = 768$ and $n = 1,000$ gave ratios 1.17886 and 1.04896. At $n = 512$ the combined method was about 21% slower than recursive contraction alone on heavy separators. These exploratory timings include sorting, lower bounds, and reconstruction; they do not establish broad superiority. The final early-rejection and two-queue implementations were also checked against the original heap certificate on all 3,000 inputs.

Optimize the baseline rather than fix it. The largest subtractable baseline need not give the strongest certificate. Set $a_0 = \min p_i$, $b_0 = \min q_j$, and use $a = a_0 - \delta$, $b = b_0 - \delta$ for an integer $0 \le \delta \le \lfloor (a_0 + b_0)/2 \rfloor$. The residual weights remain nonnegative and $a + b \ge 0$. Although one baseline term may become negative, it is only an algebraic decomposition; the input probabilities are unchanged. Let H_δ denote the residual order-free optimum. The exclusion bound is

$$B(\delta) = (a_0 + b_0 - 2\delta)(I_* + 1) + (a_0 + 2b_0 - 3\delta)n + H_\delta.$$

Every tree outside the minimum-I class costs at least $B(\delta)$. Thus any sampled price with $C_* \le B(\delta)$ certifies the same candidate.

For a fixed residual tree, increasing δ adds one unit to every key and gap weight, changing its cost at rate $2I + 3n$. Consequently B is a pointwise minimum of affine functions, hence concave. A

minimizing residual tree supplies a supergradient

$$g = 2(I - I_* - 1), \qquad B(x) \le B(\delta) + g(x - \delta).$$

If $g > 0$, smaller prices cannot improve the current value; if $g < 0$, larger prices cannot improve it. If $g = 0$, the value is globally maximal. Binary interval elimination, retaining the best sample, therefore finds the maximum over allowed integer prices when run to completion.

The same linear oracle supplies I. Give every original outcome secondary weight1, order queue items lexicographically by residual mass and number of original outcomes, and sum both merge coordinates. The first gives H_δ and the second gives $2I+3n$. This is the Huffman exchange argument with an infinitesimal positive common perturbation, choosing minimum I among residual-cost ties. The weight order is independent of δ, so no additional sorting is needed. Each price costs $O(n)$.

The fixed-profile ceiling extends to all prices: $B(\delta) \le F + a_0 + b_0 - 2\delta$. Therefore an input rejected by the existing profile test cannot be rescued by this search. The implementation tries prices only after the old certificate fails while passing that test, and permits at most $\lfloor \log_2 n \rfloor + 1$ extra oracle calls. This preserves the $O(n \log n)$ attempted-shortcut bound. Reaching the cap does not establish that the best price was found; every sampled certificate is still valid, and failure invokes the exact fallback.

The lexicographic oracle matched 400 exhaustive shape-profile comparisons through eight keys. Uncapped search matched exhaustive evaluation of all allowed integer prices on 1,200 inputs. On 3,000 additional full-OBST inputs through thirty keys, both capped fallback versions matched cubic DP and direct tree costs; 87 previously unsuccessful unrestricted certificates were recovered. A separate seeded sweep recovered the $n = 1,024$, amplitude 2 input with two additional oracle calls, and the $n = 2,048$, amplitude 1 input with three. Larger tested perturbations usually exceeded the profile ceiling. This applies standard concave-price optimization to the certificate; it is not a general subquadratic result.

In the seeded 2,048-key case, the candidate costs 43,032,446; the bound rises from 43,031,347 at $\delta = 0$ to 43,032,459 at $\delta = 125$. The three extra passes use 6,144 contractions and 6,141 pair comparisons, in addition to 4,096 shape states, 6,142 transitions, 20,784 sorting comparisons, and 2,752 original merge comparisons. Knuth uses 2,098,176 interval states and 4,198,710 root candidates on that input.

A separate six-trial comparison uses all orderings of Knuth, the old hybrid, and the price-search hybrid. All six $n = 1,024$, amplitude 2 cases and all six $n = 2,048$, amplitude 1 cases were newly certified. Median per-trial price-search/Knuth ratios were respectively 0.04953 and 0.06128. At $n = 2,048$, amplitude 2 still failed, with ratio 0.99391. Ordinary random and gap-spike controls at $n = 1,024$ gave ratios 1.06492 and 1.12941. The already-certified $n = 1,024$, amplitude 1 control made no extra oracle calls yet showed a price-search/old-hybrid ratio of 1.13564. This variability limits interpretation of small timing differences; operation counts and all raw observations are retained. The host was Apple M4 with Python 3.9.6.

7 Exact bounded-excess construction by dyadic intervals

The candidate class can be expanded beyond minimum internal path length. This is necessary even on some mildly nonuniform inputs. For $n = 255$, set all gap weights to 16 and all key weights to 16, except keys $8j + 1$ which have weight 17. The unique minimum-I tree costs 61,680; Knuth gives optimum 61,656, attained with $I = I_* + 2$. The algorithm below constructs and certifies this optimum. It retains both kinds of weight and never assumes that the restricted optimum is globally optimal.

An exact representation. An ordered full binary tree is a tiling of $[0,1]$ by consecutive aligned dyadic intervals: a gap at depth d has width 2^{-d}. A leaf starting at x must satisfy $x/2^{-d} \in \mathbb{Z}$. Conversely such a tiling is a complete prefix code and determines an ordered full tree. Alignment, not just the Kraft equality, enforces order. On a grid of height H, an interior boundary at integer coordinate Y corresponds to a key with comparison length $H - \nu_2(Y)$. Thus gap and key costs are both local in a left-to-right tiling construction.

Theorem 5 (Bounded-excess construction)**.** *For $n \geq 1$, let $N = n+1$, $h = \lfloor \log_2 N \rfloor$, $S = 2^h$, $r = N - S$, and $s = \min(r, S-r)$. For any nonnegative integer budget k, the minimum full weighted cost among all trees with $I \leq I_* + k$ can be found and reconstructed in*

$$O\big(n(s+k+1)(k+1)^2 2^k\big)$$

arithmetic operations, using $O\big(n(s+k+1)(k+1)2^k\big)$ space for backpointers.

Proof. Measure dyadic widths in units of 2^{-h} and put $w = 2^{h-d}$. For integral depth d, the potential

$$f(d) = d - h - 2 + 2^{h+1-d} = d - h - 2 + 2w$$

is nonnegative, with zeros exactly at h and $h+1$. The minimum external path length is $hN + 2r$. Since external path length equals $I + 2n$ and $\sum_i w_i = S$, every full tree satisfies

$$\sum_i f(d_i) = \sum_i d_i - (h+2)N + 2S = I - I_*.$$

Every depth in the budgeted class therefore has $f(d) \leq k$. This implies $d \leq H = h + k + 2$, and only $O(k+1)$ integral depths are possible. For integral d, direct substitution gives

$$\tfrac{1}{2} - \tfrac{1}{2} f(d) \leq w \leq 1 + f(d).$$

Apply these inequalities to a prefix of i leaves and to its remaining suffix, each having potential at most k. The cumulative position y must obey

$$\max\left(\tfrac{i-k}{2}, i - r - k\right) \leq y \leq \min\left(i + k, \tfrac{S-r+i+k}{2}\right).$$

This interval has width at most both $r + 2k$ and $(S-r)/2 + k$, hence $O(s+k)$.

Set $K = 2^{k+2}$. A state after i leaves stores integral drift $u = K(y-i)$ and spent potential $z = K \sum f(d)$. For depth d, let $W = 2^{H-d}$. Require $(iK + u) \bmod W = 0$ and update

$$u' = u + W - K, \qquad z' = z + K(d-h) + 2(W-K).$$

Retain the position band and $0 \leq z' \leq Kk$. Charge $q_i d$ and, for $i > 0$, charge $p_{i-1}(H - \nu_2(iK+u))$. Start at $(u,z) = (0,0)$ and finish after N leaves with $u = -rK$. Every retained complete path is an aligned full tiling with excess at most k; every tree in the class passes the same tests. The DP therefore optimizes exactly this class. For fixed u, the congruence $z \equiv 2u \pmod{K}$ leaves at most $k+1$ possible spent potentials. There are $O((s+k+1)2^k)$ positions per stage and $O(k+1)$ choices per state, proving the bounds. The bands admit direct array indexing; the Python prototype uses sparse dictionaries. Backpointers recover gap depths; the boundary lengths reconstruct the tree in linear time. □

Perfect-size refinement. When $N = S$, use instead $f_0(d) = d - h + \frac{3}{2}(2^{h-d} - 1)$. It is nonnegative on integer depths, vanishes only at h, and also sums to $I - I_*$. The inequality $|2^{h-d} - 1| \le 2f_0(d)$ and zero total drift imply $|y - i| \le k$. Here $H = h + k + 1$, $K = 2^{k+1}$, and $z = 2K \sum f_0$. Its increment is $2K(d - h) + 3(W - K)$, and $z \equiv 3u \pmod{2K}$. The implementation uses this smaller grid at perfect sizes, with the same asymptotic bound at $s = 0$.

Certify against all other trees. Let C_k be the computed candidate, $a = \min p_i$, $b = \min q_i$, and H' the order-free bound on residual weights. Every excluded tree has $I \ge I_* + k + 1$, so it costs at least

$$B_k = (a + b)(I_* + k + 1) + (a + 2b)n + H'.$$

Thus $C_k \le B_k$ proves global optimality. Sorting and the two-queue bound cost $O(n \log n)$ and are included in the counters. The constructor uses $k = 2$ and attempts the shortcut only when $s \le \text{bit_length}(n)$. Its attempted work is $O(n \log n)$; a failed certificate or skipped size falls back to Knuth. The total worst case remains quadratic. Arbitrary growth of k is exponential, and wide shape spaces remain quadratic even for fixed k.

Validation, failed precursor, and prior work. The implementation passed 1,200 restricted-optimum comparisons against all shapes through eight keys with budgets $0, \ldots, 4$, and 3,000 full mixed-weight checks against cubic DP. It separately checks direct tree costs, both residual-bound implementations, and exact fallback. An additional 104 family checks include zeros, geometric weights, spikes, and key-only/gap-only controls. The seeded size sweep is retained. A precursor substituting ordered depth bounds into the old certificate passed 480 exhaustive checks but rescued none of 24 near-uniform fixtures at its 32-iteration budget. This negative experiment did not prove optimal multiplier search useless; its cost and the failed perfect-tree hypothesis motivated expanding the candidate class.

On the 1,023-key periodic fixture with weights 16 and 17, the excess-two DP finds cost 312,154 and excludes other shapes with bound 312,185. It uses 27,570 states, 137,835 transition checks, 6,052 sorting comparisons, and 1,022 merge comparisons, versus Knuth's 523,776 states and 1,046,532 root candidates. At $n = 1{,}024$, a seeded amplitude-four perturbation uses 48,003 states and 240,000 transition checks, plus 13,784 sorting and 1,264 merge comparisons. It certifies cost 19,477,996.

Six matched trials per fixture use all orderings of Knuth, the preceding baseline-price hybrid, and the excess-two constructor. Median excess-two runtime divided by Knuth runtime was 0.23600 and 0.44831 on amplitude-four perturbations at $n = 1{,}023$ and 1,024, and 0.20790 and 0.17101 on the periodic fixture at $n = 1{,}023$ and 2,047. Each certified in all six trials. Ordinary random input at $n = 1{,}023$ failed and cost 1.34949 times Knuth; amplitude-sixteen input at $n = 2{,}047$ failed and cost 1.13631 times Knuth. The skipped-width control at $n = 768$ ran the same Knuth routine and measured 0.93148, illustrating noise rather than a computational gain. These are exploratory M4/Python timings including all preprocessing and reconstruction. Periodic fixtures are repeated identically; random and perturbation inputs vary between trials. All 42 observations are retained.

The dyadic/prefix-code viewpoint and sequential frontier DP are established ideas. Becker [6] gives a sequential mixed-weight DP for a near-minimal *height* restriction in quadratic time. The band above uses excess *total unweighted path length*, a different restriction; it does not improve Becker's stated problem. Priority of this particular potential, band bound, and exclusion combination remains unestablished.

8 A split-height construction covering every input size

The preceding shape band can be wide even at zero excess. We therefore implement the split-height/Monge mechanism already sketched by Karpinski, Larmore, and Rytter in Section 5 of their longer preprint [4]. That section explicitly supplies the fixed-height Monge property and the two-part construction. It is different from Section 5 of the six-page conference version [3]. No priority claim is made for this mechanism. The present experiment makes the tiling, matrix padding, reconstruction, and separate unrestricted certificate explicit.

Two scales of dyadic intervals. Fix an external height limit $H = L + U$ and write $K = 2^L$, $B = 2^U$. Precompute all relative costs $C_h(i, j)$ for $0 \le h \le L$ and $0 \le j - i \le 2^h - 1$, using

$$C_h(i, j) = W(i, j) + \min_r \{C_{h-1}(i, r) + C_{h-1}(r + 1, j)\}.$$

Empty intervals have zero cost; the root range enforces both child height capacities. Monge inequalities at fixed height give monotone minimizing roots as the endpoints move. Choose leftmost ties and bound each root by the adjacent shorter intervals. Search ranges telescope to $O(n)$ for each interval length. Summing over lengths and heights costs $O(n2^L)$.

Divide the unit dyadic interval into B top-grid blocks. Every tree of height at most H consists of lower subtrees contained in individual blocks, together with shallower gaps occupying aligned groups of blocks. Let $D_b(i)$ be the minimum partial cost after consuming i gaps and reaching grid boundary b. Keys through $i - 2$ have already been charged. For $b > 0$, put

$$R_b(i) = D_b(i) + p_{i-1}(U - v_2(b)); \qquad R_0(0) = 0.$$

The boundary key has exactly this comparison length. There are two types of transition:

$$\begin{aligned} D_{b+1}(j) &\leftarrow \min\{D_{b+1}(j), R_b(i) + C_L(i, j - 1) + UW(i, j - 1)\}, \qquad i < j \le i + K, \\ D_{b+w}(i + 1) &\leftarrow \min\{D_{b+w}(i + 1), R_b(i) + q_i(U - \log_2 w)\}. \end{aligned}$$

The second transition requires a power of two $w \ge 2$, $w \mid b$, and $b + w \le B$. Width-one gaps are zero-key lower subtrees in the first transition. Start at $D_0(0) = 0$ and finish at $D_B(n + 1)$. Dyadic alignment proves that these paths represent every height-feasible tree and only such trees. Backpointers recover every lower tree and every boundary key.

Linear matrix searches with feasible padding. At fixed b, the first transition is a banded Monge matrix in i, j; row additions and the separable interval mass preserve the inequality. Reachable counts i are contiguous: the coarsest tiling of the prefix has popcount(b) leaves, and successive refinement gives every count through bK, truncated to n before another transition. Search output columns from $i_{\min} + 1$ through $\min(n + 1, i_{\max} + K)$, each of which has a feasible input row.

SMAWK requires care outside the band. Let $S = (H + 1)(\sum p + \sum q) + 1$, larger than every feasible prefix cost. In the transposed matrix with row j and candidate column i, use

$$A(j, i) = \begin{cases} S + j - K - i, & i < j - K, \\ R_b(i) + C_L(i, j - 1) + UW(i, j - 1), & j - K \le i < j, \\ S + i - j + 1, & i \ge j. \end{cases}$$

This padding preserves total monotonicity. For $i_1 < i_2$, once i_2 is preferred at a row, it remains preferred as j increases: within the feasible band this is Monge; outside it, distances decrease toward

the band and increase away from it, and finite entries beat all padding. When the candidates straddle opposite invalid sides, their cost difference is $i_1 + i_2 + K + 1 - 2j$, also decreasing. Ties preserve the same non-strict implication. Thus SMAWK finds leftmost minima in $O(n)$ queries per grid boundary. Padding by tied infinities does not supply this argument.

Across all boundaries there are $B - 1$ aligned choices of coarse width, so the second transitions and scans also cost $O(nB)$. Choosing $L = \min(H, \lfloor H/2 \rfloor + 1)$ gives

$$O(n(2^L + 2^U)) = O(n2^{H/2})$$

work and storage including backpointers. Consequently a fixed extra height $H = \lceil \log_2(n+1) \rceil + c$ gives $O(n^{3/2})$ attempted work at *every* size. The prototype uses sparse dictionaries; the bounded integer state coordinates also admit direct array indexing.

Lemma 6 (Minimum path length with a tall path)**.** *For $1 \le R \le n$, the minimum internal path length $J(n, R)$ among n-key trees of external height at least R is obtained by forcing a path of R internal nodes and filling the remaining available positions in increasing order of depth. The excluded class is empty if $R > n$.*

Proof. Every tree in the class contains such a path, contributing $R(R-1)/2$. The remaining nodes form an ancestor-closed subset of its free forest: one side root at each depth $1, \ldots, R-1$, and two roots at depth R. The $n - R$ shallowest positions in this forest are ancestor-closed and minimize the depth sum among all subsets of that size. Their addition constructs a feasible tree attaining the bound. Directions of the forced path do not change the forest's depth counts. □

A level-count implementation computes J in $O(R + \log(n+1))$ work.

Theorem 7 (Certificate against taller trees)**.** *Let C_H be the exact height-restricted optimum above, and let Q be the order-free mixed-node Huffman lower bound for residual weights $p - a, q - b$, where $a = \min p$, $b = \min q$. If $H < n$ and*

$$C_H \le (a+b)J(n, H+1) + (a+2b)n + Q,$$

the constructed tree is globally optimal. If $H \ge n$, there is no height restriction and certification is automatic.

Proof. Every excluded tree has internal path length at least $J(n, H+1)$. Its baseline contribution is $(a+b)I + (a+2b)n$ and its residual cost is at least Q. Within the height class, C_H is optimal. □

Sorting and the residual oracle add $O(n \log n)$ work. Failed tests invoke Knuth, so the full constructor remains quadratic in the worst case. The certificate never assumes success from near-uniform weights.

Exact verification and operation evidence. The harness passed 450 exhaustive weighted height-class checks through eight keys, 44 exhaustive tall-path minima, 2,000 cubic-checked mixed inputs with complete fallback (917 successful certificates), and 104 additional adversarial-family inputs. An independent all-root height recurrence checked 520 candidates and 261,482 stored short-table entries; the same 520 cases also compared SMAWK with a retained divide-and-conquer matrix search. The larger sweep has 36 fixtures at three height budgets each. Every near-uniform fixture certified, including previously skipped sizes 384, 768, 1,000, and 1,536. All random, geometric, and gap-spike fixtures failed the certificate, sometimes despite an optimal restricted candidate.

At $n = 1{,}000$, the minimum-height candidate and full optimum are both 18,973,413, below the exclusion bound 18,974,193. The optimizer uses 117,453 short states, 220,483 short root candidates, 84,937 genuine matrix queries, 10,268 padding queries, 16,016 prefix scans, and 5,630 coarse transitions. The bound adds 10,648 sorting and 1,335 merge comparisons, plus 1,000 contractions; all keys are reconstructed. Knuth uses 500,500 states and 1,002,162 root candidates. These distinct operation types are reported separately, without treating them as equal-cost instructions.

Matched timing and failure controls. Six trials per fixture use every ordering of Knuth, the minimum-height constructor ($c = 0$), and one extra level ($c = 1$). Preprocessing and reconstruction are included; every result is checked against Knuth. Inputs vary between trials except the repeated periodic fixture. On the sponsor's M4 with Python 3.9.6, median per-trial runtime ratios were:

Fixture	$c = 0$ ratio	certified	$c = 1$ ratio	certified
384, near-uniform	1.22484	6/6	2.29965	6/6
768, near-uniform	0.94444	6/6	1.40863	6/6
1,000, near-uniform	0.71530	6/6	1.03015	6/6
1,536, near-uniform	0.65826	6/6	1.03916	6/6
3,072, near-uniform	1.46288	0/6	0.70837	6/6
1,536, amplitude four	1.67333	0/6	1.18924	6/6
1,536, ordinary random	1.55378	0/6	1.93841	0/6
1,023, periodic $A = 16$	1.69541	0/6	1.06171	6/6

All 48 observations and operation counters are retained. These are mixed practical results: small successful cases can be slower, and random cases pay substantial overhead before quadratic fallback. At 3,072 near-uniform keys, the minimum-height candidates are optimal but fail certification; one extra level strengthens the exclusion bound sufficiently. For the periodic fixture, minimum height actually excludes the true optimum. The earlier bounded-excess solver is more efficient on that narrow class. No broad practical superiority or improvement over the known width-limited Larmore/KLR constructor is established by this experiment.

9 A variable interval frontier for exact full construction

The height test can fail even when its candidate is optimal. We now use a class guaranteed in advance to contain an unrestricted optimum. The short-interval/top-skeleton architecture is established work of Larmore and Karpinski–Larmore–Rytter [2, 3]; the experiment compares a uniform width envelope with a variable interval contour. Both use the same safe mass guarantee and matrix infrastructure. This is not a priority claim for the architecture or its restricted subquadratic regime.

A failed transfer of the sharper mass constant. The initial implementation used $2W/F_{d+3}$ as the maximum internal subtree mass at edge depth d, following a literal transfer of the Fibonacci formula in [5]. Its first full-input check failed: the resulting class could be empty. The positive two-key example

$$p = (1, 1), \qquad q = (1, 100, 1)$$

explains the failure. Both trees cost 206; the total mass is 104 and the internal child has mass 102. Thus $102 > 2 \cdot 104/3$ at depth one. Charging an additional comparison at every gap is a tree-independent constant and does not remove this violation. We preserve the counterexample

and use the following independently justified, weaker constant in our model. This does not refute the KLR architecture or its asymptotic guarantees.

Lemma 8 (A sufficient mixed-weight subtree bound)**.** *If S is an internal subtree at edge depth d in an optimal full OBST of total outcome mass W, then*

$$W(S) \le W/F_{d+1}, \qquad F_1 = F_2 = 1.$$

Proof. Let X be an internal grandchild of v, with parent y, and let O be the mass inside v's subtree but outside y's, including key v. For a straight two-edge path, rotating y upward changes cost by $O - p_y - W(X)$. For a bent path, promoting X with a double rotation changes cost by $O - p_X - W(X)$. Optimality therefore implies $O \ge W(X)$ in either case. These formulas include all external gap weights.

Going upward from S, write $T_0 = S, T_1, \dots, T_d = T$. Nonnegativity gives $W(T_1) \ge W(T_0)$, and the grandchild inequality gives $W(T_i) \ge W(T_{i-1}) + W(T_{i-2})$ for $i \ge 2$. Induction gives $W(T_d) \ge F_{d+1}W(S)$. Weak inequalities include zero weights and ties. □

This lemma concerns internal subtrees. A heavy interior external gap can have nearly all the mass at depth two; zero-key subtrees remain permitted. The Fibonacci approach is prior work [5]; sharpness of this particular sufficient bound is not claimed.

Store a contour instead of a global width. For fixed d, let $E_d(i)$ be the largest $j > i$ satisfying $F_{d+1}W(i, j) \le W$, or i if there is none. Empty intervals always have stored cost zero. Nonnegative weights imply that $E_d(i)$ is nondecreasing and the domain is closed under taking subintervals. A two-pointer scan computes it in $O(n)$ work. Write

$$S_d = \sum_{i=0}^{n} (E_d(i) - i).$$

By Lemma 8, every internal subtree at depth d of an optimal tree belongs to the domain. For comparison, the uniform version sets $\ell_d = \max_i(E_d(i) - i)$ and stores

$$U_d = \ell_d(n+1) - \ell_d(\ell_d + 1)/2 \ \ge S_d$$

intervals. Both classes contain an unrestricted optimum.

Lemma 9 (Root-search work on a hereditary contour)**.** *Computing all unrestricted interval optima in this domain takes $O(n + S_d)$ work, with at most $3S_d$ tested roots.*

Proof. Bucket only existing cells by length in $O(n + S_d)$ work. Knuth's recurrence applies because every child and adjacent shorter interval is present. Let $r(i, j)$ be the leftmost optimal root and put $t(i, j) = r(i, j) - i \ge 0$. For length at least two, the root count is $2 + t(i+1, j) - t(i, j-1)$. Sum over the domain. A positive occurrence $t(a, b)$ cancels its negative occurrence whenever $(a, b+1)$ is present. An unmatched positive occurrence requires $b = E_d(a)$ and $E_d(a-1) = E_d(a)$. There is at most one per row, of size at most $E_d(a) - a$. Discard unmatched negative occurrences. Adding the length-one cells proves the $3S_d$ bound. Scanning nonexistent diagonal cells would lose this saving; the buckets are essential. □

Compose the exact interval costs. Use 2^d top-grid blocks and the preceding dyadic transitions, replacing height-limited lower costs by these unrestricted interval costs. A block starting after i gaps can finish after j exactly when $i < j \le E_d(i) + 1$. It contributes $C(i, j-1) + dW(i, j-1)$ and the starting boundary key cost $p_{i-1}(d - v_2(b))$ at $b > 0$. Coarse gaps retain their aligned power-of-two skip transitions. Every optimal tree is represented, while every reconstructed candidate is a valid full tree. Thus the resulting optimum is unrestricted.

The variable band needs another implementation detail. Reachable input rows, and their union of feasible output columns, may have holes. Scan once per top boundary to retain exactly output columns covered by some reachable row. Put $L(j) = \min\{i : E_d(i) + 1 \ge j\}$, a nondecreasing sequence with $L(j) \le j - 1$. In the transposed matrix, use finite true entries when $L(j) \le i < j$, and padding

$$Z + L(j) - i \quad (i < L(j)), \qquad Z + i - j + 1 \quad (i \ge j), \qquad Z = (n + d + 1)W + 1.$$

All finite prefix costs are smaller than Z. Within the band, exact interval costs are Monge. Across padding, opposite-side differences $i_1 + i_2 + 1 - j - L(j)$ are nonincreasing; same-side preferences also move right. Thus the matrix is totally monotone, even after removing arbitrary rows and columns, and every searched column has a feasible minimum. SMAWK, scans, and coarse transitions together cost $O(n2^d)$. Backpointers recover every key and gap contribution.

Theorem 10 (Exact mass-frontier constructor)**.** *The adaptive implementation constructs a full optimal BST in*

$$O\Big(n \log(n+1) + \min_{d \ge 0}\{S_d + n2^d\}\Big)$$

arithmetic work. The uniform-envelope implementation replaces S_d by U_d. At the selected depth, storage is $O(n + S_d + n2^d)$ or $O(n + U_d + n2^d)$ respectively. Both remain quadratic in the worst case.

Proof. The interval and composition arguments establish exactness at every depth. Start with depth zero and the full Knuth table. Evaluate successive depths while $n2^d$ is smaller than the best work proxy so far; each contour costs $O(n)$. No later depth can improve the proxy because its top term alone is too large. At most $O(\log(n+1))$ depths are evaluated. Depth zero uses Knuth directly, omitting a redundant top pass and its lower-order n proxy term. The preceding work and storage bounds now apply. □

This constructor has no posterior certificate failure followed by a second solve. It may select the full table initially. The Python prototype uses dictionaries; bounded integer top coordinates also allow direct arrays. When each key has weight at least $a > 0$ and $W = O(na)$, we have $S_d = O(n^2/\phi^d)$, where $\phi = (1 + \sqrt{5})/2$. Balancing against $n2^d$ recovers the known exponent $1 + \log_{2\phi} 2 \simeq 1.59023$.

A scoped asymptotic separation. Let n range over fourth powers and $L = n^{3/4}$. Give every key and gap weight $A = n^2$, except a central block of L keys and its $L + 1$ gaps, all of weight one. Every weight is positive and $W = \Theta(nA)$. An admitted interval contains at most $m = O(n/F_{d+1})$ heavy keys. Intervals wholly inside the light block number $O(L^2)$. Other starts contribute $O(nm)$ ordinary endpoints; only $O(m)$ outside starts can reach across a block boundary and gain $O(L)$ endpoints, and starts inside add at most $O(Lm)$ heavy endpoints. Consequently

$$S_d = O(n^2/F_{d+1} + L^2).$$

With $d = \lfloor \log_{2\phi} n \rfloor$, the adaptive work is $O(n^{1+\log_{2\phi} 2} + n^{3/2}) = O(n^{1.59024})$.

Every uniform-envelope plan, however, has proxy $\Omega(nL)$. If $2^d \geq L$, the top term suffices. Otherwise $F_{d+1} = O(L^{\log_2 \phi})$, so W/F_{d+1} exceeds the entire light block's $O(L)$ mass. Hence $\ell_d \geq L$ and the uniform table has at least $nL/2$ cells. This gives $\Omega(n^{1.75})$ at every depth. The separation is within the specified architecture, not against every prior OBST algorithm or combinations of our shortcuts. In particular, this light block also offers an opportunity for certified contraction.

Both modes were checked against Knuth on this positive family at $n = 16, 81, 256, 625$, with cubic checks also at the first two sizes. Larger instances were used only for plan counting. At $n = 65{,}536$, the chosen adaptive and uniform proxies were 119,041,540 and 361,608,522, both at depth nine. These are $S_d + n2^d$ and $U_d + n2^d$, not measured full-solve operation totals or timings. All parameter counts are retained.

Independent verification. The harness passed 960 weighted exhaustive-shape comparisons through eight keys on arbitrary nondecreasing contours, including infeasible classes. It checked 914 interval costs independently and 90,193 top minima by brute search. Another 2,400 full solves, using both modes on 1,200 inputs through 40 keys, matched cubic DP and reconstructed costs. All 24,600 internal subtrees in the cubic-optimal trees satisfied Lemma 8. Families include zeros, ties, geometric variants, spikes, and key-only and gap-only controls. A further 56 fixtures through 2,048 keys matched Knuth in both modes, including ordinary random inputs that failed the preceding height certificates.

At 2,048 random keys, the adaptive version chose depth six and returned cost 21,053,524. It used 309,261 interval cells and 616,642 root candidates, versus Knuth's 2,098,176 cells and 4,215,527 candidates. Additional work included 24,035 contour mass checks across seven parameter depths, 1,245,431 genuine matrix queries, 149,319 padding queries, 131,136 scans of each row/column type, and 107,859 coarse transitions. The uniform version stored 337,550 cells. On log-uniform weights the corresponding cell counts were 308,487 versus 431,951; on a single heavy key they were 1,048,576 versus 1,573,376. Geometric inputs selected the full table. All operation categories and parameter levels are retained separately.

Matched timings. All six orders of Knuth, uniform width, and adaptive contour are used per fixture, for 48 trials. Preprocessing and reconstruction are timed; every result is directly evaluated and checked against Knuth. Inputs vary except for deterministic geometric and spike fixtures. The timing light window uses weights near 1,000 outside a block of $\lfloor n^{3/4} \rfloor$ keys and weight one inside, with varying position; it is separate from the $A = n^2$ asymptotic family. M4/Python 3.9.6 median paired ratios were:

Fixture	uniform/Knuth	adaptive/Knuth	adaptive/uniform
512, random	1.59908	1.61204	0.96602
1,024, random	1.56592	1.55650	1.00412
2,048, random	0.90002	0.91021	1.00096
2,048, light window	1.13693	0.77546	0.65523
2,048, near-uniform	0.87926	0.91295	1.02587
1,024, log-uniform	1.67408	1.47983	0.88390
512, geometric	0.98060	1.04487	1.01772
2,048, heavy key	1.50542	1.04249	0.70113

The smaller random cases remain slower than Knuth. The contour's saving over uniform width is clearest on localized structure; ordinary random inputs show little timing difference. Both geometric variants execute the same Knuth kernel after selection, so the sub-unit ratio there is

noise. These instrumented Python measurements do not establish production-language or broad practical superiority. Every raw observation is retained.

10 Dominant-gap separators and local mass frontiers

The global mass bound can admit many unnecessary intervals when a single heavy outcome dominates otherwise ordinary regions. We first split at certified outcomes, then apply the frontier constructor with each terminal component's own total mass. The following exchanges retain all key and gap weights. Their priority is not established.

Lemma 11 (Restriction to an ordered interval)**.** *A BST restricted to a contiguous key interval and its endpoint gaps has no greater depth for any retained outcome. If the old root is outside the retained key interval, every retained outcome loses at least one comparison.*

Proof. Delete each comparison at an outside key and follow the branch containing all retained outcomes. Comparisons at retained keys remain. This gives the required BST, with each retained search path a subsequence of its old path. The argument also applies to an empty key interval containing one gap. □

Fix an interior gap g in a current component. Its adjacent keys are $a = g - 1$ and $b = g$. Let L contain the keys strictly left of a, with its endpoint gaps, and let R contain those strictly right of b with its endpoint gaps. Their masses are W_L, W_R, and the component mass is

$$W = W_L + p_a + q_g + p_b + W_R.$$

Theorem 12 (Interior-gap separator)**.** *If*

$$q_g + \min(p_a, p_b) \;\geq\; W_L + W_R, \tag{3}$$

there exists an optimal tree with gap g at depth two. Its top two keys are a, b, in either orientation, and its remaining subtrees are L, R. Writing C_L, C_R for their unrestricted optimal relative costs gives

$$\mathrm{OPT} = C_L + C_R + W + q_g + \min(p_a + W_L,\; p_b + W_R). \tag{4}$$

At equality, other optimal trees need not have this shape.

Proof. Take any tree in which g has depth at least three. Restrict it to L, R using Lemma 11. If its root lies in L, make a the new root, attach restricted L on its left, and make b its right child, with g on b's left and restricted R on its right. Every L outcome deepens by at most one. Every R outcome lost the old root comparison, then gains two, so also deepens by at most one. Key a saves at least one comparison, key b does not deepen, and gap g saves at least one. The cost change is at most $W_L + W_R - p_a - q_g$. For an old root in R, the mirror exchange costs at most $W_L + W_R - p_b - q_g$.

If the old root is a, keep it and promote b to its right child. The L depths do not increase and R depths increase by at most one. As a, b are adjacent and g was deeper than two, b had depth at least two and saves at least one comparison. The change is at most $W_R - p_b - q_g$. The case with old root b is symmetric. Condition (3) makes each bound nonpositive, so the exchange can be applied to an optimum. An interior gap cannot have depth below two; at depth two its bounding ancestor keys must be exactly a, b.

Both orientations leave the same two independent optimization problems. With root a, the cost is $C_L + C_R + W + q_g + p_b + W_R$; with root b it is $C_L + C_R + W + q_g + p_a + W_L$. Minimize these two values after solving each outside interval once. This proves (4) without duplicated recursive branches. □

Replacing the smaller neighboring weight in (3) by the larger one is false. For $p = (100, 1, 50)$ and $q = (1, 1, 1, 1)$, the weakened test accepts $g = 1$ since $101 \geq 53$. Yet the full optimum is 212 and forcing this gap to depth two costs 261. The valid test rejects it since $2 < 53$. This strictly positive counterexample is retained.

Endpoint and key certificates. For a component's first key and left endpoint gap, $2(p_i + q_i) \geq W$ suffices to put that key at the root in some optimum; the right endpoint is symmetric. If its original depth is $h \geq 1$, its endpoint gap has depth $h+1$. Promotion saves $h(p_i+q_i)$ while restricting and reattaching the other interval increases cost by at most $W - p_i - q_i$. Likewise, $2p_r \geq W$ certifies an arbitrary key r as root: promote it and attach the two restrictions, increasing every other depth by at most one while saving at least one comparison on r. These are sufficient conditions, not a general largest-weight root rule.

Avoiding quadratic separator scans. Components remain contiguous intervals of the original input, with no weight aggregation. Store the static scores

$$G_j = 2q_j + p_{j-1} + p_j + \min(p_{j-1}, p_j), \qquad K_j = 2p_j.$$

An interior gap in component $[i, k)$ satisfies (3) exactly when $G_j \geq W(i, k)$, for $i < j < k$. Two range-maximum segment trees find qualifying gaps and keys in $O(\log(n+1))$ work per query, after $O(n)$ construction. Prefix sums give component masses in constant work. Each separator removes one or two keys, and terminal intervals are disjoint. Only terminal slices are copied, so a long chain does not cause quadratic copying or rescanning. The implementation tests endpoints first and builds the indexes only when an endpoint test fails on both sides. All-zero components are returned as balanced trees directly.

Theorem 13 (Local-frontier composition)**.** *For terminal components t with m_t keys and local admissible-interval counts $S_{t,d}$, the constructor is exact and uses*

$$O\left(n \log(n+1) + \sum_t \min_d \{S_{t,d} + m_t 2^d\}\right)$$

arithmetic work. If $m_t \leq b$ for every terminal, the bound is $O(n \log(n+1) + nb)$. Sequential reuse of terminal scratch storage gives $O(n + \max_t \{S_{t,d_t} + m_t 2^{d_t}\})$ space at selected depths d_t. The unrestricted worst case remains quadratic.

Proof. The separator identities reduce the optimum to disjoint exact subproblems. There are $O(n)$ separator calls and $\sum_t m_t \leq n$. The local frontier bounds apply separately, including parameter selection since $\sum_t m_t \log(m_t + 1) \leq n \log(n+1)$. Each terminal may select depth zero and use its full Knuth table, so $\sum_t m_t^2 \leq nb$ when all sizes are at most b. Include the reconstructed output and separator stack in $O(n)$ space; completed terminal scratch arrays need not remain allocated. □

Two exact structured regimes. For $p_i = q_i = 2^i$ and $q_n = 2^n$, the total mass is $3 \cdot 2^n - 2$ and the rightmost key/gap pair has mass $3 \cdot 2^{n-1}$. The right endpoint certificate passes, and the remaining prefix has the same form. The optimal chain is constructed with n endpoint decisions, no range index, and no terminal DP: $O(n)$ arithmetic work. At the final single key the implementation selects the equivalent left endpoint rule. Weights have $O(n)$ bits, so this is not a linear bit-complexity assertion.

For a different positive mixed family, recursively plant a heavy central gap separating two outside intervals, stopping below 32 keys. Set each planted weight to four times its component mass after its children have been planted. Each planted gap satisfies the theorem, and all terminal sizes are below 32. This gives $O(n \log n)$ arithmetic work on the family, including varying positive background weights on both keys and gaps. Neither regime is an unrestricted subquadratic result.

Verification and complete work accounting. The harness exhaustively checks 22,140 weight assignments in $\{0, 1, 2\}$ through four keys against every tree shape. It checks 3,211 qualifying gap certificates and their cost identities, 2,036 endpoint certificates, and 380 key certificates. A further 3,280 full solves, using both frontier and Knuth terminal strategies on 1,640 inputs through 40 keys, match independent cubic costs and direct tree evaluation. Inputs include empty, zero, tied, geometric, spike, key-only and gap-only controls. Both strategies and the direct global frontier also match Knuth on 40 fixtures through 2,048 keys.

At $n = 2048$, the direct and local frontier interval counts are:

Fixture	direct frontier	local frontier	terminal components
Random	308,115	308,115	1
Near-uniform	309,290	309,290	1
Geometric	2,098,176	0	0
Heavy key	1,048,576	153,660	2
Heavy gap	1,047,552	153,582	2
Random background, heavy gap	1,047,552	153,362	2
Hierarchical gaps	522,242	22,333	64

Cell counts do not include all work. On hierarchical gaps, 63 gap separators require 254 endpoint checks, 6,142 index-build comparisons and 1,476 range query comparisons. Terminal solves use 42,885 root candidates, 39,284 genuine top-matrix queries, 6,261 padding queries, 8,122 contour mass checks, 6,952 scans of each row/column type, and 1,792 coarse transitions. Their 1,922 keys are reconstructed and relabeled. On the heavy-gap fixture, 305,118 terminal root candidates and 1,214,092 genuine top queries accompany the reduced interval table. All counter categories are retained; metadata such as local selected depths and actual heights are summed over terminals, not reported as a global tree height. Geometric input uses 2,048 endpoint separators and 4,095 endpoint checks, with no index or terminal work.

Matched timings and competing shortcuts. Ten balanced execution orders compare five constructors per fixture: Knuth, direct frontier, separator with frontier terminals (SF), separator with Knuth terminals (SK), and the earlier recursive contraction solver (RC). These are the five cyclic rotations and their reversals, not all 120 permutations. The 70 trials include 350 timed solves, preprocessing and reconstruction. Every reconstructed cost is checked and all five answers agree. Inputs vary between trials except deterministic spike and geometric families. M4/Python 3.9.6 median paired runtime ratios to Knuth are:

Fixture	frontier	SF	SK	RC
2,048, random	0.87985	0.87172	1.03209	1.00381
2,048, near-uniform	0.87747	0.90863	1.06833	1.06450
2,048, heavy key	0.99362	0.74038	0.52307	0.01605
2,048, heavy gap	0.99617	0.77197	0.51357	1.03871
2,048, random heavy gap	0.90780	0.75211	0.49766	0.97692
2,048, hierarchical gaps	1.06953	0.04608	0.01843	0.99421
1,024, geometric	1.05318	0.00781	0.00548	0.04206

The new gap certificates improve regimes where the earlier contraction solver does little. They do not replace that solver: RC is much faster on the heavy-key fixture. Local frontier construction also costs more than plain Knuth on these smaller terminal components despite storing fewer cells; SK is faster than SF on every separator-active fixture. The geometric SF/SK paths execute the same endpoint logic with no terminal solve, so their timing difference does not reflect an algorithmic distinction. Random and near-uniform fixtures have no separator and expose screening overhead. All raw observations are retained. These instrumented Python results establish scoped empirical benefits, not broad production-language superiority or an asymptotic speedup inferred from timing.

A positive obstruction to the combined constructor. For $n \geq 5$, set every key and gap weight to one except $q_1 = q_{n-1} = M = n^3$. Then $W = 2M + 2n - 1$. The largest gap score is $2M + 3 < W$, endpoint scores are four, and key scores are two, so no separator fires. Indeed the frontier planner selects depth zero and the full table. To see this, write $\tau(k) = k(k+1)/2$ and $h = \lfloor (n-1)/2 \rfloor$. Depth one admits all $\tau(n)$ cells. At depth two, an interval containing one heavy gap has mass $M + 2\ell$, so the cutoff $W/2 = M + n - 1/2$ admits it exactly for length $\ell \leq h$. There are four disjoint sets of h such intervals, and $\tau(n-4)$ intervals avoiding both heavy gaps. Thus

$$S_2 + 4n = \tau(n-4) + 4h + 4n = \tau(n) + 6 + 4h > \tau(n).$$

For $d \geq 3$ with $2^d \leq n$, we have $3 \leq F_{d+1} \leq 2^d \leq n$. The cutoff is below M but above every all-unit interval mass, so $S_d = \tau(n-4)$ and

$$S_d + n2^d \geq \tau(n-4) + 8n = \tau(n) + 4n + 6 > \tau(n).$$

If $2^d > n$, the top term alone exceeds $n^2 \geq \tau(n)$. Every positive depth is therefore worse than the direct Knuth plan. This is a quadratic obstruction to the specified constructor, not a general OBST lower bound. An independent contour check rejected the initial assumption that all one-heavy-gap intervals were admitted at depth two; the correct length cutoff above is retained in the test. The family motivates joint treatment of several comparable heavy outcomes. Eight full instances through 256 keys match Knuth and reconstructed costs, with cubic checks through 32 keys. Four larger instances through 65,536 keys check plans only; at that size the selected table has 2,147,516,416 cells, not a completed full solve.

11 Joint heavy-outcome cores

The two-gap obstruction motivates selecting several heavy outcomes together. The following certificate uses their necessary comparison keys as an ancestor-closed core. It preserves the complete mixed-weight objective and solves every intervening interval exactly. Priority of the sufficient condition has not been established.

For $n \geq 1$, choose a nonempty set $\mathcal{H}$ of positive-weight key or gap outcomes. Let A contain every selected key and the adjacent key(s) of every selected gap. Write $m = |A|$, $\theta = \min_{x \in \mathcal{H}} w_x$, and let W_0 be the total weight outside $\mathcal{H}$. Both adjacent keys of an interior gap are ancestors of that gap in every BST. Thus each A-key lies on the search path to some selected outcome, possibly itself. Call a tree *core-closed* if no key outside A is an ancestor of an A-key. Then A forms a top BST and the intervals between consecutive A-keys, including both end intervals, are its payloads. Selected gaps remain singleton core gaps.

Lemma 14 (Projection preserving selected depths)**.** *Every BST can be reduced to a BST on A without increasing any selected outcome's depth. If the original tree is not core-closed, at least one selected outcome becomes strictly shallower.*

Proof. Consider an outside key r whose subtree contains an A-key. If just one child contains A-keys, replace r by that child. The other branch contains no selected outcome: any nonempty subtree containing a selected gap must contain one of its neighboring keys, and a selected empty child gap would make r itself an A-key. All retained selected outcomes save a comparison.

If both children contain A-keys, let a be the largest such key in the left subtree. Replace r by a, restrict the old left subtree to keys below a and their endpoint gaps, and retain the old right subtree. No selected outcome lies strictly between a and r: a selected key would belong to A, and a selected gap would put an adjacent key in A before or at r. All A-keys survive. Selected outcomes in the left restriction lose zero or more comparisons by Lemma 11; right depths are unchanged. If a is selected, it moves strictly upward. Otherwise it must border a selected gap on its left. Its right gap cannot be selected, since that would put $a+1$ in A, contradicting either the maximality of a or $r \notin A$. The selected left gap loses the comparison at the old a. Thus some selected depth decreases in either case.

These exchanges may discard light keys, which will be reinserted later. Each exchange reduces the number of outside keys and retains A. Repeat until no outside key has an A-descendant, then discard the remaining anchor-free branches. Selected gaps are preserved throughout. This produces the asserted core tree. Arbitrary deletion of nonretained comparisons would not suffice: a nonretained key can separate retained outcomes on both sides; the explicit promotion above resolves that case. □

A first sufficient condition is $\theta \geq (m+L)W_0$, where $L = \lceil \log_2(n+1) \rceil$. Project an optimum and reinsert balanced payloads: the selected cost falls by at least θ, and the new light cost is at most $(m+L)W_0$. The next theorem substantially sharpens the height term using the earlier Fibonacci lemma.

Theorem 15 (Protected-height core certificate)**.** *Let $W_{\mathcal{H}}$ be the selected total weight and*

$$H = \max\{j \in \{1, \ldots, m\} : F_j \theta \leq W_{\mathcal{H}}\}, \qquad F_1 = F_2 = 1.$$

If

$$\theta \;\geq\; (H+L)W_0, \tag{5}$$

there exists a core-closed optimum. If the payload intervals have masses W_t and unrestricted relative optimum costs C_t, then

$$\mathrm{OPT}(\textit{original}) = \mathrm{OPT}(\textit{original A-key weights},\ (W_t)_t) + \sum_t C_t.$$

Equality in the certificate does not force every optimum to be core-closed.

Proof. In a protected-only core instance, keep selected weights and set every other weight to zero. Every internal subtree contains an A-key and therefore a selected outcome of weight at least θ in that subtree. By Lemma 8, a depth-d internal subtree of a protected-only optimum has mass at most $W_{\mathcal{H}}/F_{d+1}$. Its height, including external leaves, is consequently at most H.

Project a non-core-closed full optimum by Lemma 14, saving at least θ in selected weighted cost. Replace the projected core by a protected-only optimum; its selected weighted cost cannot increase, although individual depths may change. Reinsert balanced payloads. Every unselected key comparison length or gap depth is at most $H+L$, so the new light cost is at most $(H+L)W_0$. The old light cost was nonnegative. Condition (5) makes the exchange nonincreasing, establishing the existence of a core-closed optimum.

At core-gap depth d, an exact payload contributes $dW_t + C_t$. The core and payload relative costs therefore optimize independently, proving the identity. Empty intervals contribute zero relative cost and keep their gap weight. The final full-weight core is optimized without a height restriction: H bounds the protected-only witness, not the output core. □

For k equal selected weights, $H = O(\log(k+1))$, whereas m can be $2k$. The certificate thus handles much less extreme dominance than the size-based version. It does not remove unselected weights from the final objective.

A threshold sweep with no repeated interval scans. The implementation selects all outcomes of weight at least a threshold θ. Key i is in A precisely when $\max(p_i, q_i, q_{i+1}) \geq \theta$. Sort these activation weights and all outcome weights. Sweep thresholds upward, grouping equal values. Before each threshold, activate as unanchored each key whose activation weight is strictly smaller. A disjoint-set structure maintains the contiguous payload runs and their total triangular state count. A singleton adds one; merging runs of sizes a, b adds ab. Union by size bounds each find by $O(\log n)$, even without using an inverse-Ackermann analysis.

At each certified threshold, evaluate

$$\tau(m) + \sum_t \tau(k_t), \qquad \tau(k) = k(k+1)/2.$$

Keep the minimum proxy, accepting only a strict improvement on $\tau(n)$. This optimizes a Knuth-state proxy, not actual frontier runtime. Fibonacci values through index n take $O(n)$ additions. Since $W_{\mathcal{H}}/\theta$ is nonincreasing during the upward sweep, a single decreasing Fibonacci index supplies the height bound in $O(n)$ total index decrements; cap it by m. The entire selection uses $O(n \log(n+1))$ arithmetic work and $O(n)$ arithmetic storage. Failure invokes the preceding separator constructor, adding screening work without a failed quadratic solve.

Kernels, reconstruction, and scoped bounds. Both core and payloads use either exact local frontiers or Knuth. Before each kernel, uniform key weights p and uniform gap weights q are recognized. Such a tree costs $(p+q)I + np + 2nq$, where I is its unweighted internal path length. A recursively balanced tree minimizes I and is built in linear work. This familiar uniform-weight shortcut is not claimed as new.

If $K(s)$ bounds the chosen exact kernel on s keys, successful construction costs

$$O\left(n \log(n+1) + K(m) + \sum_t K(k_t)\right), \qquad m + \sum_t k_t = n.$$

Sequential scratch reuse retains only the payload outputs between solves. The general work and space bounds remain quadratic. A uniform residual background and k selected outcomes give $m \leq 2k$ and work $O(n \log(n+1) + k^2)$. If residual weights lie in $[a, Ca]$ for fixed $a > 0$ and C, each payload's local frontier costs $O(s^\alpha)$, where $\alpha = 1 + \log_{2\phi} 2 < 1.59024$. A certified plan then costs

$$O\left(n \log(n+1) + k^2 + \sum_t k_t^\alpha\right).$$

With payload sizes at most B, the last sum is at most $nB^{\alpha-1}$. These are bounds for the certified plan; minimizing the triangular proxy need not minimize this sharper expression.

For the preceding obstruction, $q_1 = q_{n-1} = n^3$ with unit background, $n \geq 5$, we have $m = 4$, $H = 3$, and $W_0 = 2n - 1$. The certificate passes, so the four-key core and uniform payloads

take $O(n \log n)$ total work. The previous separator/frontier constructor used all $n(n+1)/2$ cells. Bounded positive mixed background and sufficiently large equal selected gap weights instead give an $O(n^{1.59024})$ plan. This resolves that scoped obstruction; it is not an unrestricted subquadratic theorem.

Independent checks and operation counts. Every nonempty selected-outcome subset and every tree shape through four keys were examined. In all 1,937 non-core-closed cases a core-closed tree has componentwise no-greater selected depths and at least one strict improvement. On 22,140 inputs with weights in $\{0, 1, 100\}$, all 17,054 qualifying nontrivial certificates match the optimum over every shape and the composition identity. Another 2,640 full solves on 1,320 inputs through 32 keys, with both kernels, match independent cubic DP and reconstructed costs. On 520 small inputs the threshold sweep matches direct enumeration of every threshold and payload partition. Both kernels also match Knuth and the previous separator constructor on 30 larger fixtures through 2,048 keys.

At $n = 2048$, the following cell counts include both core and payload solves, or the complete fallback when certification fails:

Fixture	previous separator	joint core/frontier	core keys
Random	309,201	309,201	—
Near-uniform	309,290	309,290	—
Two heavy gaps, unit background	2,098,176	10	4
Two heavy gaps, random background	2,098,176	306,506	4
45 heavy gaps	304,486	23,965	90
45 moderate gaps	304,486	23,695	90
45 mixed heavy outcomes	305,421	24,016	68

The moderate-gap fixture sets each selected weight to exactly $(H + L)W_0 = 42{,}395{,}178$, with $H = 9$, $L = 12$, and $W_0 = 2{,}018{,}818$. The old size-based condition would require $(90 + 12)W_0$ and fails. Work includes 63,549 sorting comparisons, 2,047 Fibonacci additions, 3,024 height checks, 1,958 activations, 1,912 unions, and 1,511 find steps. Kernels use 45,365 root candidates, 116,338 genuine top queries, 23,904 padding queries, 12,336 contour checks, and 17,488 scans each for rows and columns. All 2,048 keys are reconstructed. Local kernel metadata such as heights and chosen depths are sums across calls, not global tree heights. The uniform two-gap case still pays selection, 2,044 payload-node constructions and relabelings, and ten core DP cells; it does not perform only ten total operations.

Matched construction timings and controls. Ninety matched trials compare five solvers in ten balanced orders per fixture: five cyclic rotations and their reversals, not all 120 permutations. All 450 timed solves include preprocessing and reconstruction; their costs agree and every output tree is directly evaluated. CF and CK denote the new core with frontier and Knuth kernels, respectively; S is the preceding separator solver and RC the earlier recursive contraction solver. M4/Python 3.9.6 median paired runtime ratios to Knuth are:

Fixture	S	CF	CK	RC
2,048, random	0.85643	0.86589	1.00373	1.02785
2,048, near-uniform	0.90215	0.85239	1.02368	1.04692
2,048, two gaps, unit background	0.97393	0.00700	0.00551	0.98506
2,048, two gaps, random background	1.01846	0.70676	0.79129	0.99945
2,048, 45 heavy gaps	0.84243	0.08905	0.03125	0.99792
2,048, 45 moderate gaps	0.81550	0.08573	0.03229	0.98979
2,048, 45 mixed heavy outcomes	0.87777	0.08823	0.03204	1.00118
512, random	1.68034	1.76764	1.03732	1.02439
1,024, geometric	0.00648	0.02255	0.02378	0.03135

The new core resolves cases where both earlier solvers remain near Knuth's cost. Plain Knuth is faster on the many small payloads, while the frontier helps on the large random payload. The uniform two-gap CF/CK paths use the same uniform shortcut and tiny core, so their timing difference is not evidence of an asymptotic distinction. Controls expose limitations: the 512-key random frontier path is slower than Knuth, and joint threshold screening makes the geometric case slower than the existing endpoint-only solver. Failed-core controls execute the same separator kernel; sub-unit paired ratios there do not demonstrate an algorithmic benefit. These instrumented Python measurements do not establish production-language or broad practical superiority. An earlier size-certificate timing batch was explicitly stopped after the height refinement; its incomplete timings are not used here.

Full construction beyond the quadratic-table scale. On the unit-background two-heavy-gap family, full trees were constructed at $n = 512, 2048, 8192, 32768, 65536$. Each exact cost matched an independent cubic solve of the four-key instance

$$p = (1, 1, 1, 1), \qquad q = (1, n^3, 2n - 7, n^3, 1),$$

plus $2I_*(n-4) + 3(n-4)$ for the uniform payload, and direct evaluation of the full reconstructed tree. The uniform closed form was separately checked against cubic DP through 16 keys. At $n = 65536$, the exact cost is 1,407,374,885,846,941. The full new solve took 0.59933 seconds in one exploratory run, with ten core DP cells and fifteen core root candidates. It also used 196,799 sorting comparisons, 65,535 Fibonacci additions, 65,535 height checks, 65,532 activations, 65,531 unions, 65,529 find steps, and 196,609 uniformity comparisons. All 65,532 payload keys were constructed and relabeled, and the four core keys were reconstructed.

The previous method was only planned at this size: it selects 2,147,516,416 cells, and its full solve was not executed. Hence no measured runtime ratio to that solve is claimed. The construction result, the separate plan count, and the arithmetic complexity theorem are distinct evidence. Unbounded integer weights and Fibonacci values also preclude interpreting the arithmetic bound as an established bit-complexity bound.

12 Residual-cost certificates for heavy-outcome cores

The preceding height certificate charges all light search cost against one saved heavy comparison. Every competing tree must itself pay for light searches. Crediting a valid lower bound for that unavoidable cost gives a stronger test for a fixed selected set. This is an additive exclusion certificate built on Lemma 14; it does not assert a new Huffman algorithm or an established priority claim.

Write the original nonnegative weight vector as $w = w_H + w_0$, where w_H retains selected positive outcomes and is zero elsewhere. Let A be the necessary core keys from Section 11, let P be the

exact protected-only optimum on that core, and let B_0 be any valid lower bound on the full n-key problem with residual weights w_0. In particular, the order-free mixed-node relaxation of Section 15 supplies B_0. Residual weights on both keys and gaps are retained.

Theorem 16 (Residual exclusion certificate)**.** *Every tree that is not core-closed has full cost at least*

$$P + \theta + B_0, \qquad \theta = \min_{x \in \mathcal{H}} w_x.$$

Consequently, if a core-closed witness has full cost $U \le P + \theta + B_0$, some global optimum is core-closed. More strongly, if the exact optimum C_A within the core-closed class satisfies this inequality, then C_A is the unrestricted optimum.

Proof. Project any non-core-closed tree T onto A. Its selected weighted cost decreases by at least θ, and the projected selected cost is at least P. Thus $C_{w_H}(T) \ge P + \theta$. Independently, $C_{w_0}(T) \ge B_0$. Add the inequalities, using linearity of tree cost in the weights. A witness at or below this excluded-class lower bound cannot be worse than every non-core-closed tree. Optimizing within the core-closed class therefore gives a global optimum. Equality allows both classes to contain optima. □

A test before solving the payloads. Solve the protected core, retaining one optimal shape, and attach balanced payloads. This produces a full witness of cost $P + U_0$, where U_0 is its exact residual cost. The inexpensive sufficient condition becomes

$$U_0 - B_0 \le \theta. \tag{6}$$

Only after it passes does the default constructor solve the full-weight core and all payloads exactly. Thus failed witnesses need no exact payload tables. For the same selected set, this test includes the preceding height certificate: $U_0 \le (H + L)W_0$ and $B_0 \ge 0$. The implemented threshold selection is not exhaustive, so this logical implication is not a claim that the new selector always chooses the old selector's successful set.

An optional refinement solves one core-closed candidate even after its witness fails and tests $C_A \le P + \theta + B_0$. For example, $p = (0, 0, 1)$, $q = (0, 4, 1, 1)$, and $\theta = 4$ give $P = 8$, $B_0 = 4$, and excluded-class lower bound 16. The first witness costs 17, while the exact closed candidate costs 14 and is globally optimal. A failed witness is not evidence that the core structure is invalid.

Several candidates without quadratic screening. A single minimum-proxy threshold can include ordinary outcomes whose weights are too small to pay for the exchange. The revised selector keeps two representatives in each dyadic core-size band: the least $\tau(m) + \sum_t \tau(k_t)$ proxy and the highest threshold. It considers only $m \le b = \lfloor \sqrt{n \lceil \log_2(n+1) \rceil} \rfloor$ and only strict proxy improvements over $\tau(n)$. The existing sorted activation sweep finds these representatives in $O(n \log(n+1))$ work. There are $O(\log(n+1))$ candidates, and their squared core sizes sum to $O(b^2)$, because at most two lie in each geometrically increasing band.

Sort the original key and gap profiles once. For a threshold θ, remove the profile entries at least θ and append the same number of zeros. The remaining profile is still sorted. The mixed-node two-queue bound then takes $O(n)$ work per candidate, without another sort. Building and evaluating each balanced witness also takes $O(n + m^2)$ work. All candidate screening therefore costs $O(n \log(n+1))$ arithmetic operations. Candidates are tried in increasing proxy order until a witness succeeds. If none succeeds, the optional mode refines only the first candidate; limiting refinement to one avoids an $O(\log n)$ factor on failed exact payload solves. Failure invokes the preceding exact constructor.

A successful plan retains the bound

$$O\left(n\log(n+1)+K(m)+\sum_t K(k_t)\right).$$

Both modes keep quadratic general worst-case work and storage. The witness screen can use $O(n\log n)$ storage for protected-core tables; the final kernel can still require quadratic space. Ordinary Huffman coding is classical [9]; the mixed-node relaxation and its limitations are already described in Section 15. The result here is the certified use of a residual lower bound against a specified excluded class, rather than a relaxation substituted for the full objective.

A linear-weight separation from the height certificate. Let $n=2^h\ge 8$, put every original key and gap weight equal to one, and replace q_1,q_{n-1} by an equal selected weight $\theta>1$. The necessary core has four keys. Its protected optimum is $P=5\theta$: each selected interior gap needs at least two comparisons, and both cannot have depth two because their adjacent-key pairs are disjoint. Depths two and three are attainable.

Use the core whose root is original key 1, left child is key 0, right child is key $n-2$, and right-right child is key $n-1$. The sole nonempty payload contains $n-4$ keys and hangs at depth two. The residual cost of this balanced-payload witness is

$$U_0=(2h+3)n-6h-3.$$

Indeed, the core's residual cost is $4n-1$, while $I_{\min}(n-4)=(h-2)n-3h+5$ gives payload cost $2I_{\min}(n-4)+3(n-4)=(2h-1)n-6h-2$.

For the residual problem all key weights are one, all ordinary gap weights are one, and the two selected gaps have weight zero. The identity $E=I+2n$ rewrites any residual tree cost as $\sum_j(q_j+1)d_j-n$. The order-free leaf weights are therefore two ones and $n-1$ twos. Huffman first merges the two ones; the resulting $n=2^h$ weights are all two and have a perfect code of height h. Hence the exact order-free residual lower bound is

$$B_0=2nh-n+2,\qquad U_0-B_0=4n-6h-5.$$

Thus $\theta\ge 4n-6h-5$ suffices for core closure. In particular, $\theta=4n$ passes, while the preceding height condition requires $\theta\ge(h+4)(2n-1)$ and rejects it.

At $\theta=4n$, the described core is also optimal with the full collapsed weights. The four protected-optimal shapes have residual core costs $4n-1$ or $6n-9$; every other core shape has selected cost at least $6\theta>5\theta+4n-1$. It follows that the full optimum is

$$\mathrm{OPT}=(2h+23)n-6h-3. \tag{7}$$

The new constructor uses a constant-size core and a uniform payload, for $O(n\log n)$ work and $O(n)$ space on this strictly positive mixed-weight family. These weights and the construction's intermediate values have $O(\log n)$ bits. This is a separation from the preceding height certificate, not a quadratic lower bound for its fallback frontier or for other algorithms.

Independent exact checks. On all 22,140 assignments in $\{0,1,4\}$ through four keys, every one of 17,054 nontrivial threshold cases passed the excluded-class lower-bound check against all tree shapes. There were 13,669 passing witness tests and 15,181 passing exact-candidate tests, including 1,512 cases rescued by refinement. Another 14,912 excluded-tree checks used arbitrary

selected subsets, weights in $\{1, 3\}$, and up to three keys. These latter selections need not be weight thresholds.

Both kernel choices and both refinement settings passed 6,560 full solves on 1,640 inputs through 40 keys against independent cubic DP and direct tree evaluation. All 1,640 candidate portfolios matched direct enumeration of the two representatives in each core-size band. Forty-eight larger fixtures and nine additional fixtures with independently varying positive key and gap weights matched Knuth through 2,048 keys. The varying backgrounds are integers in $[1, 1000]$ and their selected outcomes have weight $4000n$; the unit-background success rows below use weight $4n$. These are finite tests, not an assertion that every such random instance passes certification.

At $n = 2048$, selected operation categories are:

Fixture	Previous intervals	New intervals	Protected proof cells
Two linear-weight gaps, unit background	568,350	10	10
45 linear-weight gaps, unit background	304,486	1,394	4,095
45 mixed outcomes, unit background	305,421	1,250	2,346
Two gaps, varying background	951,851	307,541	30,941
45 gaps, varying background	304,486	23,757	22,519
45 mixed outcomes, varying background	305,421	24,163	27,558
Random control	308,860	308,860	33,380

New intervals count all executed full-weight kernel cells, including fallback, but exclude the separately displayed protected-core proof cells. The proof column sums all attempted cores. Selected-state metadata is not counted again as executed cells. Sorting, profile scans, lower-bound contractions, witness construction, kernel matrix work, and reconstruction have separate raw counters; this table is not a total instruction count. The random control fails all ten witnesses and pays additional proof work before fallback.

Matched timings, including failures. Eighty matched trials compare four complete constructors in eight balanced orders per fixture: four cyclic rotations and their reversals, not all 24 permutations. All 320 solves include preprocessing and reconstruction; all output costs agree and are directly checked. The default witness mode is timed. On M4/Python 3.9.6, median paired runtime ratios to Knuth are:

n, fixture	Previous core/frontier	New/frontier	New/Knuth
1,024, two gaps, $4n$	2.70316	0.01539	0.01965
1,024, two gaps, $2n$	2.14252	2.12346	1.08928
1,024, many gaps, unit background	1.21361	0.03458	0.03469
1,024, mixed outcomes, unit background	1.47715	0.03594	0.03472
1,024, random	1.58933	1.66642	1.17777
1,024, near-uniform	1.47283	1.51952	1.06257
512, geometric	0.05133	0.24322	0.29896
1,024, two gaps, varying background	3.16889	1.23537	0.87326
1,024, many gaps, varying background	1.53202	0.15569	0.10279
1,024, mixed outcomes, varying background	1.50427	0.14830	0.11509

The structured successes coexist with losses on ordinary random inputs. The geometric control is about 4.91 times slower than the previous core/frontier solver because many failed witnesses precede its inexpensive endpoint fallback. Near-uniform inputs have no candidate within the cap, but still pay for selection. Sub-unit changes on failed-certificate rows must not be interpreted as an algorithmic gain. These instrumented Python measurements do not establish production-language superiority.

Full large constructions and a remaining certificate gap. For every $n = 2^h$, $3 \le h \le 16$, the full $\theta = 4n$ construction matches Equation (7), direct evaluation of the entire output tree, and an independent four-key cubic core plus the uniform payload formula. At $n = 65,536$, the optimum is $3,604,381$. The proof core and full-weight core each use ten cells and fifteen root candidates. Other counts include 196,799 selection-sort comparisons, 131,204 bound-sort comparisons, 65,536 bound contractions, 65,535 two-queue comparisons, 65,532 activations, 65,531 unions, and 65,529 find steps. All 65,536 witness keys are constructed and evaluated, then all output payload and core keys are constructed. The lower bound is $3,604,482$, giving a certificate margin of 101. The full solve took about 0.6574 seconds once, an exploratory timing. The previous method was only planned at this size: it rejects the core and selects 146,498,625 frontier cells at depth 11. No runtime ratio to that unexecuted full solve is claimed.

Failure remains informative. At $n = 2048$ and selected gap weight $2n$, the exact core-closed cost $71,611$ equals the unrestricted optimum, but the exclusion lower bound is only $67,586$; even refinement cannot certify it. At weight n, the closed candidate costs $61,371$ while the unrestricted optimum is $59,332$. Thus some failures reflect a weak certificate and others reflect a genuinely suboptimal proposed structure. Improving the bound must preserve this distinction.

Raw results and source hashes are in `core_exclusion*.json` under `obst/results`. An initial full-harness failure was a metadata bug: list-valued fallback fields were passed to arithmetic counter update. Direct assignment fixed it, and the final full harness was rerun. A completed single-threshold experiment was superseded after it missed several useful cores; the published selector and every final result use the bounded portfolio. The general subquadratic objective remains unresolved.

13 Coupled two-gap skeleton certificates

The preceding certificate adds independent protected and residual minima. We now retain enough geometry to couple these costs at the first possible violation. Select internal gaps $a < b$, with weights M_1, M_2, and let $A = \{a-1, a, b-1, b\}$, $m = |A| \in \{3, 4\}$, and $\theta = \min(M_1, M_2)$. A tree is A-closed when no outside key is an ancestor of an A-key. All weights below may be zero.

Lemma 17 (First violating skeleton)**.** *In a nonclosed tree the selected gap depth sum D is at least $m+2$. If $D = m+2$, the union of the two search paths has exactly $m+1$ keys: the keys in A and one outside key. The paths share only the root.*

Proof. Each key neighboring a gap is an ancestor of that gap. Thus the path union contains A. Nonclosure puts an outside ancestor of an A-key in the same union. If $L \ge 1$ keys are shared, its size is $D - L$, so $D \ge m + 1 + L \ge m + 2$. Equality forces $L = 1$ and union size $m + 1$. □

The one outside key is called the bridge. Its inorder position is in one of the light intervals between keys of A. Enumerating binary-tree shapes on A and the bridge takes at most $C_5 = 42$ shapes per interval. Only nonclosed shapes with selected depth sum $m + 2$ are retained.

Lemma 18 (Unordered mixed forests)**.** *Let $F_k(P, Q)$ be the minimum mixed cost of a k-root binary forest with roots at depth zero, internal-weight multiset P, and leaf-weight multiset Q, where $|Q| = |P| + k$. Weights may be permuted within each type and empty trees are allowed. Sort P ascending. For each weight in that order, merge the two smallest current leaf weights with it, add the resulting weight to the cost, and reinsert it as a leaf. After all merges the accumulated cost is $F_k(P, Q)$.*

Proof. Choose a deepest internal node in an optimal forest; it is a cherry. Exchange weights to put a smallest internal weight there and the two smallest leaf weights at its children, without increasing cost. Contract the cherry to a leaf of their total weight. Its removed cost is exactly that total. Conversely, expanding this leaf in any feasible contracted forest has that added cost. Induction proves the recurrence, including forests consisting only of leaves. Nonnegative weights ensure the generated merge weights are nondecreasing. Two queues therefore take linear work after sorting, for total $O(n \log n)$ work. □

Delete A from inorder to obtain $m+1$ intervals. Two are the selected singleton gaps. For every other region s, write its weights as (P_s, Q_s) and mass as W_s. In a retained skeleton, let h_s be its hole depth. An ordinary region contributes the lower bound

$$h_s W_s + F_1(P_s, Q_s).$$

If the bridge is in region t, there are two holes. Lower their depths to their minimum h. If the bridge comparison length is ℓ, then $h \geq \ell$, since both holes descend from it. All other internal slots in this relaxed region have comparison length at least $h+1$. Exchanging internal weights therefore places a largest regional weight x at the bridge. The relaxed contribution is

$$\ell x + h(W_t - x) + F_2(P_t \setminus \{x\}, Q_t). \tag{8}$$

This exchange applies after lowering the two hole depths; it does not identify the bridge key in the original ordered tree. Keep the exact costs of all A-keys and both selected gaps. Minimizing the sum of these costs and the regional bounds over retained skeletons gives B_{first}, a valid lower bound for $D = m + 2$.

Let B_0 be the unordered mixed-tree optimum after zeroing the two selected gap weights, retaining all other weights. Each internal gap has depth at least two, so define

$$B_{\text{basic}} = 2(M_1 + M_2) + (m-2)\theta + B_0,$$
$$B_{\text{tail}} = 2(M_1 + M_2) + (m-1)\theta + B_0.$$

The first bounds every nonclosed tree; the second bounds those with $D \geq m + 3$.

Theorem 19 (Coupled exclusion certificate)**.** *Every nonclosed tree has cost at least*

$$B = \max\{B_{\text{basic}}, \min(B_{\text{first}}, B_{\text{tail}})\}.$$

If the first class is empty, replace the inner minimum by B_{tail}. An A-closed witness of cost $U \leq B$ certifies that some unrestricted optimum is A-closed. Its exact construction then requires independent exact solutions of the intervening intervals and a constant-size core. The screening work is $O(n \log n)$.

Proof. Lemma 17 partitions excluded trees into the first and tail classes. Each has its stated lower bound, and both have B_{basic}. The witness is no more costly than any excluded tree. An optimum of the closed class is consequently globally optimal. There are constantly many regions, forest computations, and skeletons; Lemma 18 gives the screening bound. □

The implementation first uses midpoint payloads and an exact full-weight core as the witness. Optional refinement computes the exact closed candidate when this witness fails. An uncertified attempt falls back to the preceding exact method; the general worst case remains quadratic.

13.1 A family with arbitrary nonuniform key positions

Constant key weights require a scope qualification. For $p_i = c$, the internal/external path-length identity $E = I + 2n$ gives

$$C(T; c, q) = \sum_g (q_g + c) d_T(g) - cn.$$

Such inputs reduce to alphabetic leaf-weight problems. The next family instead has a linear number of nonconstant key weights.

Theorem 20 (Nonuniform-key family)**.** *Let $n = 2^h$, $h \geq 5$, $r = n/16$, and $s = r - 1$. All gaps have weight one except $q_1 = q_{n-1} = M = 3n$. All keys have weight one except any s keys in positions $2, \ldots, n-3$, which have weight two. The midpoint witness passes Theorem 19 for every placement of these keys. Combining it with the existing mass-frontier construction takes $O(n^\alpha)$ arithmetic work, where $\alpha = 1 + \log_{2\varphi} 2 < 1.59024$. For $h \geq 6$, the earlier fixed-pair bound B_{basic} cannot certify even the exact closed optimum on this family.*

Proof. For unit keys write $U_0 = (2h+3)n - 6h - 3$. The middle payload has $n - 4$ keys, mass $2n - 7$, and optimal relative cost $(2h-1)n - 6h - 2$. The minimum four-key core has selected depth sum five, middle hole depth two, and other contribution thirteen, giving cost $5M + 4n - 1$. For completeness, the distinct other triples (selected depth sum, middle depth, other contribution) among its 14 shapes are $(5,3,12), (6,3,13), (6,3,15), (6,4,14), (7,4,13)$. None is cheaper. Thus the unit-key closed optimum is $5M + U_0$. A midpoint payload attains that baseline cost and has maximum relative key comparison length h, so the nonuniform witness satisfies

$$U \leq 5M + U_0 + (h+2)s. \tag{9}$$

Let $G_k(t)$ be the minimum sum of comparison lengths of t marked internal nodes in a k-root forest. Filling the capacities $k2^{\ell-1}$ at comparison lengths $\ell = 1, 2, \ldots$ gives

$$G_1(s) = (h-5)r + 1, \qquad G_2(s-1) = (h-6)r + 2.$$

In the unit first-violation relaxation the middle two-root forest has $n-5$ internal nodes. Its balanced profile has three leaves at depth $h-2$ and $n-6$ at depth $h-1$. The forest identity $E = I + 2(n-5)$ yields

$$F_2 = (2h-3)n - 6h + 5.$$

There are ten admissible five-key skeleton profiles. Their distinct triples (minimum middle hole depth d, bridge length ℓ, fixed core/endpoint contribution c) are

$$(2,2,16),\ (2,2,15),\ (3,3,13),\ (2,1,15),\ (3,1,14),\ (2,1,16).$$

Their unit costs are $6M + 2d(n-4) + \ell + c + F_2$; the minimum at $(2,1,15)$ is

$$B^{\text{unit}}_{\text{first}} = 6M + (2h+1)n - 6h + 5.$$

With nonuniform keys the bridge takes weight two in the relaxation. The remaining $s-1$ extra units receive hole shift at least two and forest comparison lengths summing to at least $G_2(s-1)$. The bridge adds at least one. Therefore

$$B_{\text{first}} \geq B^{\text{unit}}_{\text{first}} + 2s - 1 + G_2(s-1).$$

The residual unordered optimum is exactly

$$B_0 = 2nh - n + 2 + G_1(s).$$

For the unit baseline, absorbing the key weights leaves two leaf weights one and $n-1$ weights two. Their Huffman tree is a perfect h-level tree with one bottom leaf expanded into the two unit leaves; subtract n to recover mixed cost $2nh-n+2$. Every tree pays at least $G_1(s)$ for the extra key units. The baseline optimal shape has its top $h-4$ internal levels full, so these same slots attain $G_1(s)$ simultaneously in the unordered problem, establishing equality.

Substitute $M = 3n$ in these bounds and in (9):

$$B_{\text{first}} - U \geq 5n/8 + h + 9 > 0, \qquad B_{\text{tail}} - U \geq 25n/16 + 7h + 8 > 0.$$

The witness certifies every placement. The remaining middle interval has key weights in $[1, 2]$ and unit gap weights. Its bounded-positive mass-frontier bound is $O(n^\alpha)$, and the $O(n \log n)$ screening and full reconstruction fit within this bound. This exponent is inherited from that architecture, not a new unrestricted exponent.

Finally, every closed tree costs at least its unit-background closed optimum plus $G_1(s)$. Since $B_{\text{basic}} = 6M + B_0$, its deficit is at least $n - 6h - 5 > 0$ for $h \geq 6$. This is a separation from the earlier certificate for this fixed pair, not a claim about every alternative core its selector could attempt. □

The family's total mass is $O(n)$ and all key weights are at least one. Consequently the earlier global mass frontier already has the same $O(n^\alpha)$ guarantee on the entire instance. Theorem 20 is a certificate separation with arbitrary nonuniform key placements, not an extension of the known subquadratic input regime. Its practical value must be judged by actual construction work and matched measurements.

13.2 Verification and measured limits

Independent forest enumeration passed 11,880 checks. Exhaustive small weight inputs numbered 24,188, with 51,654 pair-exclusion checks and separate checks for both first and tail lower bounds. Among these are 3,288 exact closed candidates accepted by the coupled bound but rejected by B_{basic}. Complete construction passed 6,560 comparisons with independent cubic DP, covering both payload kernels and refinement modes. Larger tests at $n = 128, 512, 2048$ include random, near-uniform, geometric, single-spike, adjacent-pair, interior-pair, and varying-weight inputs. Neither favorable examples nor these exhaustive finite checks replace the preceding proofs. Raw counts and source hashes are stored in `pair_skeleton*.json`.

The nonuniform-family harness checks 31 placement/size combinations, including clustered, spread, and random placements; 28 are compared with Knuth and eight with cubic DP. All satisfy the exact residual formula and the proved certificate margins. Fourteen unit-background cases extend through $n = 65,536$. At $n = 2048$ with unit keys and selected gap weight $2n$, the earlier bound $67,586$ is replaced by $71,619$, which certifies the exact cost $71,611$. At gap weight n, the closed candidate remains suboptimal and is correctly rejected.

At $n = 16,384$ the seeded nonuniform instance has 1,023 doubled key weights. Full construction returns cost $767,240$, with witness $768,946$, first bound $780,208$, and tail bound $795,651$. Its 7,758,912 short states, 3,922,465 top states, and 20 total core states give 11,681,397 DP states versus 134,225,920 in a full Knuth table. Parameter-selection metadata is not counted a second time. There are also 42,351,013 top-matrix queries, 15,502,473 short-root candidates, 261,181 forest

sorting comparisons, 49,143 forest merges, and 256,673 contour mass checks. The complete tree is reconstructed and its cost evaluated directly. This run took about 54.59 seconds once; the largest instance is certified by the theorem and construction, not compared with an independently executed quadratic table.

Balanced matched measurements at $n = 512$ use the four cyclic orders of Knuth, the previous residual constructor, and the new constructor with each of its two payload kernels, together with their reversals. All 224 timed complete constructions agree in cost within each trial and pass direct tree evaluation. The table reports median paired time ratios to Knuth; lower is faster. PF and PK denote the new frontier and Knuth payload kernels. The varying background uses independent weights in $[1, 1000]$ and selected gaps $1000n$.

Input	Previous/Knuth	PF/Knuth	PK/Knuth
Nonuniform family, spread	3.13537	1.48227	0.93990
Nonuniform family, random	2.99217	1.39273	0.86098
Unit background, gaps $2n$	2.77283	0.04285	0.03927
Varying background	2.86574	3.01630	1.30401
Random control	1.95412	2.01031	1.43424
Near-uniform control	1.77849	1.89871	1.28003
Geometric control	0.34365	0.30066	0.33002

Only the first three families pass the new certificate in these timed trials. In particular, the new frontier route is still slower than plain Knuth on nonuniform $n = 512$ inputs, despite taking about 48% of the previous constructor's time. The Knuth payload kernel gives a modest improvement there but retains quadratic payload work. Ratios below one on failed-certificate controls reflect the fallback and measurement variation, not successes of the new certificate.

A separate larger check uses four alternating execution orders per size and fresh random key placements in the nonuniform family. All 16 complete constructions agree with exact Knuth and direct tree evaluation. At $n = 2048$ and 4096, the median new/Knuth time ratios are respectively 0.65948 and 0.56309, about $1.52\times$ and $1.78\times$ faster. These are instrumented Python measurements on the sponsor's M4, not comparisons against optimized compiled implementations. Raw trials and source hashes are in `pair_skeleton_crossover.json`.

The chief unresolved issue is how to retain useful path-union information when no constant-size core can be certified. The present method does not bound the number of necessary boundary states for arbitrary mixed input. Its general worst-case construction remains quadratic.

14 Exact checks and preliminary measurements

The code uses nonnegative integer weights, an independent cubic recurrence, Knuth's recurrence, and direct evaluation of reconstructed trees. The basic harness additionally enumerates all tree shapes for small sizes. CPU experiments ran on the sponsor's M4 Mac mini. The reported quantities are algorithmic counts, not inferred asymptotic bounds from elapsed time.

- Single-key contraction: 22,143 exhaustive weight assignments in $\{0, 1, 2\}$ through $n = 4$, and 6,000 seeded random instances through $n = 20$. Every cost and reconstructed tree matched the cubic oracle.
- Block theorem: 19,486 qualifying interval contractions across 4,500 seeded instances through $n = 16$ matched independent cubic costs.

- Block algorithm: 3,000 random instances through $n = 20$, including fixed and adaptive thresholds, matched the cubic oracle and tree evaluation. Additional small cases check threshold selection against every threshold.

Table 1 records one seeded instance per family at $n = 512$. The baseline has 131,328 states. The block column includes all states inside selected blocks, not just the core. Results from different solvers use their separately recorded seeds and are not paired timing experiments.

Table 1: Preliminary exact DP-state counts, including block solves.

Family	Single-key pass + DP	Adaptive blocks + DP
Uniform mixed weights	131,328	131,328
Near-uniform mixed weights	131,328	131,328
Random mixed weights	128,778	130,817
Log-uniform weights	116,886	94,964
Keys only (control)	92,235	88,508
Sparse heavy separators	—	10,344

In the heavy-separator instance all gaps and ordinary keys have integer weights in $\{1, 2, 3\}$, and 22 randomly selected keys have weight 10^6. The block algorithm retains exactly 22 keys. Its state reduction is about 12.7-fold, but this is not a claim of a 12.7-fold runtime speedup. On ordinary random mixed weights the gain is small. The single-key pass contracts every key for the tested sequence $p_i = 4^i, q_i = 4^i$, but not for most of the ratio-two sequence. Geometric integer inputs also illustrate why bit complexity must be distinguished from arithmetic-operation complexity.

15 A useful relaxation that failed as a general solver

Allow arbitrary assignment of key weights to internal positions and gap weights to external positions. In this relaxed problem, repeatedly take the two smallest current external weights a, b and the smallest remaining internal weight p, replace them by $a + b + p$, and add that sum to the cost.

To prove the rule, take a deepest internal node of an optimum. Its children are leaves. Swapping the smallest internal weight into it and the two smallest external weights into its children cannot increase cost. Contract that cherry and induct. Sorting and a heap give $O(n \log n)$ time. This elementary extension of the Huffman exchange argument is not claimed as novel.

The relaxed cost is a valid lower bound for full OBST. Equality with a feasible ordered tree certifies optimality. It agreed with an independent shape-enumeration and rearrangement oracle on 22,143 exhaustive and 2,700 random instances in the basic harness.

However, using the relaxation on demand in an exact sparse DP was poor on random inputs. At $n = 128$ it visited roughly 4,300–4,600 exact intervals instead of Knuth's 8,256, but spent approximately 3.9–4.1 million weight comparisons constructing bounds and inspected over 100,000 candidate roots. Fewer exact states did not mean less computation.

Nor does the relaxation's interval matrix inherit the required quadrangle inequality. For $p = (9, 1, 12)$, $q = (5, 1, 18, 10)$, its costs H satisfy

$$H(1, 2) + H(2, 3) = 60 > 54 = H(1, 3) + H(2, 2).$$

Thus applying Knuth-style matrix optimization directly to this bound would be unjustified.

Two additional exact counterexamples exclude tempting local certificates. With $p = (3, 5, 18)$, $q = (13, 14, 4, 8)$, the tree with root 2 and left subtree $0(\varnothing, 1)$ costs 127, every single-rotation neighbor costs at least 134, yet the balanced tree costs 125. With $p = (18, 8, 18)$, $q = (17, 11, 1, 13)$, the balanced tree costs 164 and every distinct tree obtained by promoting a descendant to the root of an existing subtree costs at least 169; the tree $0(\varnothing, 2(1, \varnothing))$ costs 157. These are barriers to those local certificates, not complexity lower bounds.

16 An ordered depth-chain lower bound

Let l_i be the comparison length of key i and g_i the depth of gap i. For every valid BST, adjacent keys have different lengths and

$$g_0 = l_0, \quad g_n = l_{n-1}, \quad g_i = \max(l_{i-1}, l_i) \quad (0 < i < n).$$

This identity is existing work; see Becker [6], Lemma 1. Let K_d count keys of length d and G_d count gaps at depth d. The root and child-count identities are

$$K_1 = 1, \qquad G_d + K_{d+1} - 2K_d = 0 \quad (1 \le d \le n), \qquad K_{n+1} = 0.$$

Permit any sequence in $\{1, \dots, n\}^n$ with unequal adjacent entries. For arbitrary multipliers $\lambda_0, \dots, \lambda_n$, minimize

$$\sum_i p_i l_i + \sum_i q_i g_i + \lambda_0(K_1 - 1) + \sum_{d=1}^{n} \lambda_d(G_d + K_{d+1} - 2K_d). \tag{10}$$

Call the minimum $L(\lambda)$. Every actual tree is permitted and has zero penalty, so $L(\lambda) \le \mathrm{OPT}$ for every multiplier vector. The expression is a chain with key energy $U_i(b) = p_i b + \lambda_{b-1} - 2\lambda_b$, gap energy $V_i(b) = q_i b + \lambda_b$, and constant $-\lambda_0$. For an interior transition into length b, take the smaller of

$$\min_{a<b} F(a) + V_i(b), \qquad \min_{a>b}\{F(a) + V_i(a)\}.$$

Prefix and suffix minima give $O(H)$ work per key for a domain of H lengths. Using $H = n$ would cost quadratic time even for one bound evaluation.

Theorem 21 (Exact affine-tail compression)**.** *Fix $h \ge 1$, arbitrary $\lambda_0, \dots, \lambda_h$, and a slope $0 \le s \le \min_i p_i$. Extend $\lambda_d = \lambda_h + s(d - h)$ for $d > h$. The minimum in (10) is unchanged if lengths are restricted to $1, \dots, \min(n, h + 3)$.*

Proof. For $b \ge h + 1$, the unary key energy has slope $p_i - s \ge 0$ and the gap energy has slope $q_i + s \ge 0$. Both are nondecreasing in this tail. If some $l_i > h + 3$, replace it by a member of $\{h + 1, h + 2, h + 3\}$ unequal to its current neighbors. At most two values are forbidden. The key energy does not increase. Each adjacent maximum is either unchanged or decreases within the affine tail, so neither gap energy increases. Endpoint gaps obey the same argument. Repeat for all large lengths. The restricted domain is a subset of the original domain, and every original sequence has a no-more-expensive restricted sequence; therefore the minima are equal. □

Taking $h = \lceil \log_2(n+1) \rceil + 2$ gives an $O(n \log n)$ oracle. This is a statement about minimization of the relaxed energy, with no height assumption on the unknown optimal BST. Optimizing only the head multipliers requires folding all tail constraint violations into the derivative for λ_h. The

implementation uses exact integer multipliers and costs scaled by 1024. Its subgradient ascent is heuristic; the bound is valid at every iteration regardless of convergence.

A stronger oracle requires exactly one length1 key. For each candidate root, the left and right chains then contain only lengths at least2. One forward and one reverse pass supply their closing costs for every root position, giving all forced-root bounds in $O(nH)$ total time. The projection above preserves the unique root, so tail compression remains valid for each marginal. Retaining the maximum bound seen for each root, then taking the minimum over roots, is also a valid lower bound.

Both oracles passed 50 comparisons against exhaustive length-sequence minimization, 5,000 arbitrary-multiplier bound checks against cubic OBST, and 1,050 direct comparisons of the full and shortened oracles. Forced-root marginals were additionally checked against exact root-constrained costs.

A structural gap. The sequence $l = (3, 1, 2, 3, 2)$ has $g = (3, 3, 2, 3, 3, 2)$, a unique root, and all the level-count equalities. It is not a valid BST depth sequence. For

$$p = (3, 27, 21, 16, 12), \qquad q = (18, 22, 27, 22, 12, 22),$$

it costs 470, while the true optimum is 472. Its penalty is zero for every multiplier vector. Hence neither oracle family can ever exceed 470 on this instance. More optimization iterations cannot remove this obstruction.

Preliminary strength and cost. At $n = 256$, normalized gaps $(\mathrm{OPT} - \mathrm{LB})/W$ for the chain bound versus order-free Huffman were 0.15003 versus 0.22080 on a random mixed instance, and 0.00873 versus 0.49294 on an alternating instance. However, 300 ascent iterations used 1,082,368 key-depth cells including initialization, compared with 32,896 interval states for Knuth. The unique-root variant uses two passes and still leaves 254 of 256 potentially improving roots on the random instance, and 255 on the alternating instance. Only two survive on the gap-spike instance. These results identify a lower-bound component and its limitations; they do not establish a faster general exact solver.

17 Exact depth ordering and an arbitrary-window obstruction

Call a positive integer sequence *ranked* if every nonempty contiguous interval has a unique minimum. Equivalently, equal labels have a smaller label between them. This is vertex ranking on a path with the label order reversed; vertex ranking and bounded-path variants are existing notions [7, 8]. The following results explain the ordering information absent from the preceding relaxation; no novelty claim is made for this characterization.

Theorem 22 (Depth characterization)**.** *A sequence $l \in \{1, \ldots, n\}^n$, $n \geq 1$, is the inorder key-comparison-length sequence of a BST if and only if it is ranked, $K_1 = 1$, and every level-flow equality holds, with gaps derived from l as above.*

Proof. Necessity follows from the unique shallowest key in each inorder interval and the two children of every internal node. For sufficiency, delete all labels at most d. Let U_d count the remaining nonempty runs, and let A_d count the deleted labels. If the deleted labels form R runs and include e endpoints, then the number of gaps of length at most d is $A_d - R + e$, whereas $U_d = R + 1 - e$. Thus

$$G_{\leq d} = A_d + 1 - U_d.$$

Summing the level-flow equalities from 1 through d, and using $K_1 = 1$, also gives

$$G_{\le d} = A_d + 1 - K_{d+1}.$$

Hence $U_d = K_{d+1}$. Ranking permits at most one label $d+1$ per remaining run, so equality forces exactly one. Starting with the unique length1 root, every remaining nonempty interval therefore receives its root at the next level. This constructs the tree with exactly the supplied lengths. □

Ranking has a linear recognizer: scan with an increasing stack, pop larger labels, reject an equal top, and otherwise push. A rejected pair has no smaller intervening label; conversely every such pair causes rejection. Each entry is pushed once and popped at most once. This establishes cheap recognition, not cheap minimization over all valid sequences.

Theorem 23 (Bounded-window obstruction)**.** *For every integer $w \ge 2$ and $t \ge 2$, there is a strictly positive full-OBST instance with $n = w + 3$ keys and an inadmissible depth sequence satisfying all level-flow equalities, a unique length1 root, and ranking on every interval of at most w keys. Its cost is at most $(7/8 + 1/(8t))\,\mathrm{OPT}$. Consequently every lower bound obtained by pricing the level-flow equalities over this window-constrained domain has the same upper limitation, regardless of the multipliers.*

Proof. Take

$$l = (3, 1, 2, 3, 4, \ldots, w+1, 2).$$

The ascending middle portion has $w - 1$ entries. Its two enclosing length2 labels form the only interval with a repeated minimum, of width $w+1$. Every shorter window passes. Direct substitution into the gap formula verifies $K_1 = 1$ and every level-flow equality. The sequence is invalid: the unique root is key1, but its singleton left subtree has length3, and two keys to its right both claim length2.

Give every gap weight1 and every key weight1 except

$$p_1 = 3B, \qquad p_2 = p_{n-1} = B.$$

The unit-weight outcomes contribute $E = w^2 + 4w + 10$ in the inadmissible sequence, whose cost is $7B + E$. Every actual tree costs at least $8B$. If key1 is the root, the two B-weight keys lie to its right and their comparison lengths sum to at least5, giving $3B + 5B$. Otherwise key1 has length at least2 and each B-weight key has length at least1, giving $6B + 2B$.

Set $B = tE$. The inadmissible cost satisfies

$$7B + E = \left(\frac{7}{8} + \frac{1}{8t}\right) 8B \le \left(\frac{7}{8} + \frac{1}{8t}\right) \mathrm{OPT}\,.$$

Its level-flow penalty is zero for every multiplier vector, so every priced minimum is at most this cost. □

For $t = 16$ the upper ratio is $113/128$, independently of w; as t increases it approaches $7/8$. This rules out universal exactness of the specified window-minimum repair, not other local constraints, adaptive nonlocal cuts, or subquadratic full-OBST algorithms. Equivalently, at every $n \ge 5$ the construction passes all windows of up to $n - 3$ keys.

Burying the obstruction below a forced prefix. Let a current instance have mass W and a feasible tree of cost U. Prepend a key of weight $A = W + U + 2$ and a new leftmost gap of weight1. Using the new key as root costs $A + 1 + W + U = 2A - 1$, while every other root costs at least $2A$ from that key alone. Hence the new root is forced and $\mathrm{OPT}_{\mathrm{new}} = A + 1 + W + \mathrm{OPT}_{\mathrm{old}}$. The transformation $l \mapsto (1, l_0 + 1, \ldots, l_{n-1} + 1)$ preserves zero flow and local window ranking, and changes the inadmissible cost by the same $A + 1 + W$. Repeating this D times places the strict gap beneath the true optimal top-D right-chain prefix. Thus fixing any constant number of top levels cannot make the remaining window/flow relaxation exact. The constant relative gap is not claimed after this embedding, because the forcing weights add a potentially large common cost.

Independent verification. The harness enumerated all 873,612 sequences in $\{1, \ldots, n\}^n$ for $1 \leq n \leq 7$. The characterization accepted exactly the independently generated BST depth sequences: respectively 1, 2, 5, 14, 42, 132, and 429. The stack recognizer agreed with direct interval scans on every sequence. Width3 and exact flow already admit four invalid sequences at $n = 6$ and 34 at $n = 7$. The positive family was checked at $t = 16$ for all 127 widths $2 \leq w \leq 128$: exact costs, window validity, zero flow, and the tree-cost lower bound all passed. Knuth and direct tree evaluation checked every instance; independent cubic DP also checked $w \leq 24$. Another 120 cases combine $2 \leq w \leq 16$ with forced prefix depths $1 \leq D \leq 8$. Cubic DP, Knuth, direct costs, and inspection of the optimal prefix verify the exact additive identity and preserved strict gap.

18 Endpoint responses: insufficiency and representation size

An intact subtree interacts with its ancestors through its relative cost and total mass. If its interior is not yet fixed, one tempting richer summary is the response to added weights at its endpoint gaps:

$$\mathcal{R}(x, y) = \mathrm{OPT}(p, q + xe_0 + ye_n), \qquad x, y \geq 0.$$

For a fixed tree the response is $C(T) + xd_0(T) + yd_n(T)$, so $\mathcal{R}$ is a concave lower envelope of affine planes. We distinguish whether this function contains enough information for composition from how many pieces an explicit representation requires. Neither result below is a general lower bound for optimal-BST construction; priority remains unestablished.

18.1 Equal responses need not be interchangeable

Theorem 24 (Insufficient endpoint-value interface)**.** *An interval's endpoint response, length, total mass, and endpoint gap weights do not determine its optimal cost in an arbitrary surrounding search problem. This remains true if its full vector of key weights and its multiset of gap weights are also retained.*

Proof. Consider two four-key intervals with

$$p = (1, 1, 2, 3), \quad q^A = (1, 4, 2, 1, 1), \quad q^B = (1, 4, 1, 2, 1).$$

Both have mass 16 and endpoint gap weights one. Enumerating the 14 four-key shapes gives the following least intercepts for each possible endpoint-depth pair:

Depths	A	B
$(1,2)$	45	45
$(1,3)$	42	42
$(1,4)$	44	45
$(2,1)$	43	43
$(2,2)$	36	36
$(2,3)$	36	37
$(3,1)$	39	39
$(3,2)$	36	36
$(4,1)$	43	42

Each plane with slopes $(1,4), (2,3), (3,2)$, or $(4,1)$ is pointwise dominated on the nonnegative quadrant by a common plane whose slopes and intercept are no larger. Hence both responses are exactly

$$\mathcal{R}(x,y) = \min\{45 + x + 2y,\ 42 + x + 3y,\ 43 + 2x + y,\\ 36 + 2x + 2y,\ 39 + 3x + y\}.$$

This proves equality for all parameters, not just sampled ones.

Prepend keys of weights $(7,1)$, append keys of weights $(1,7)$, and give the four new gaps weight one. The full inputs are

$$P = (7,1,1,1,2,3,1,7),$$
$$Q^A = (1,1,1,4,2,1,1,1,1),$$
$$Q^B = (1,1,1,4,1,2,1,1,1).$$

Using the exact recurrence, the least full costs conditional on each root are

Root	0	1	2	3	4	5	6	7
A	108	108	103	101	101	101	111	110
B	108	108	103	101	100	101	111	110

Thus the optima are 101 and 100. Direct enumeration of all 1,430 eight-key trees independently verifies both rows. Equal summaries with the identical surrounding context therefore have unequal answers. □

In particular, exact prepend/append updates closed on the stated summary cannot exist: identical starting summaries and identical updates would remain indistinguishable. The theorem does not exclude extra shape information, non-exposed depth states, signed-price responses, or richer interfaces. It does not assert a distinction between mixed and alphabetic algorithms; it rejects this summary even on small positive mixed inputs.

The lost information includes internal alternatives with identical boundary depths. Both intervals have minimum cost 36 at depths $(2,2)$. The shape rooted at key 2, with left child 0 (whose right child is 1) and right child 3, also has depths $(2,2)$, but costs 37 in A and 36 in B. The least intercept erases this distinction; the surrounding tree can make the additional alternative valuable.

18.2 Quadratic explicit envelopes with linear factorizations

For any $n \geq 1$, the endpoint paths share only the root. If their depths are a, b, their union has $a+b-1 \leq n$ keys. At most one least intercept per depth pair is needed, giving at most $n(n+1)/2$ affine pieces. The quadratic order can occur already for a single interval.

Theorem 25 (Explicit-facet obstruction)**.** *There are strictly positive mixed-weight instances on $n = 2m+1$ keys whose endpoint response has at least m^2 affine pieces on the nonnegative quadrant. Consequently, explicitly listing all such pieces can require $\Omega(n^2)$ entries.*

Proof. Set $K = 4m$ and $P_i = K8^i$ for $0 \leq i < m$. First consider these m keys with all gaps one and added left endpoint weight x. Write $S_j = \sum_{i=0}^{j} P_i$ and $x_j = 2S_j + 2m$.

An endpoint key and its adjoining outer gap are forced to the root if their combined weight exceeds the mass of all other outcomes. Indeed, each rotation of that key toward the root saves one comparison on both outcomes and adds at most one to each other outcome. The strict inequality makes every rotation improve cost. At x_j, for every $k > j$,

$$P_k + 1 > S_{k-1} + x_j + k + 1,$$

while in the remaining prefix

$$P_0 + 1 + x_j > (S_j - P_0) + (j+1).$$

The geometric series and $K = 4m$ imply these inequalities. For example, $S_{k-1} < P_k/7$ and $x_j \leq 2S_{k-1}+2m$ for $k > j$, whereas $P_k \geq 32m$. Thus every optimum first uses roots $m-1, m-2, \ldots, j+1$, then key 0, giving left endpoint depth $m - j$. All m possible one-sided slopes occur, with strict inequalities persisting in open neighborhoods of the probes.

Mirror this instance about a central key:

$$p = (P_0, \ldots, P_{m-1}, R, P_{m-1}, \ldots, P_0), \quad q = (1, \ldots, 1).$$

Let $X = x_{m-1}$, $S = S_{m-1}$, $B = 2S + 2m + 2 + 2X$, and $R = (2m+2)B$. For $0 \leq x, y \leq X$, the mass outside the central key is at most B. A center-root tree costs at most $R + nB < 2R$; any other root charges the central key at least $2R$. Thus the center root is forced. At (x_j, x_k), every optimum has endpoint depths

$$(m - j + 1, m - k + 1).$$

These are m^2 different gradients. Since there are finitely many trees and all minimizing trees at each probe have the same gradient, that probe has an open affine neighborhood. Different gradients require different pieces. □

The weights have $O(n)$ bits each. This is an arithmetic/output-size obstruction to explicit facet enumeration, not a word-RAM lower bound with $O(\log n)$-bit weights, nor a lower bound for a single optimum query. The same example also explains why counting facets cannot rule out implicit representations.

Lemma 26 (Exact hinge factorization)**.** *For the one-sided family above, let*

$$C_0 = \sum_{i=0}^{m-1} (m-i)P_i + m + \frac{m(m+1)}{2}.$$

Its response for every $x \geq 0$ is

$$H_m(x) = C_0 + mx - \sum_{i=1}^{m-1} \max\{0, x - (P_i - P_0)\}.$$

Within the forced-center box the two-sided response equals

$$R + 2S + 2m + 2 + x + y + H_m(x) + H_m(y).$$

Proof. At x_j, the remaining right subtree of key 0 is a decreasing-key chain, by the same strict endpoint condition. Passing from j to $j+1$ rotates key 0 above key $j+1$. At base weights the intercept increases by $P_{j+1} - P_0$, while the endpoint slope decreases by one. At $j = 0$ the intercept is C_0. Every possible endpoint depth occurs among these m trees, and optimality at a probe proves its intercept is minimal for that depth. Their successive intersections are the increasing thresholds $P_i - P_0$, giving the hinge formula. The center-root recurrence gives the two-sided identity. □

The factorization extends to the entire quadrant. Set

$$W = R + 2S + 2m + 2, \qquad V = W - P_0 - 1,$$
$$L = C_0 - mP_0 - m - 1, \qquad J = R + 2(S - P_0) + 2m + 2L.$$

Here L is the fixed cost of the remaining one-sided keys $1, \ldots, m-1$, and J is the fixed inner cost after both outer keys and outer gaps are removed.

Theorem 27 (Full-quadrant factorization)**.** *For the geometric family and every $x, y \geq 0$,*

$$\begin{aligned}\mathcal{R}(x,y) = W + x + y + \min\{&H_m(x) + H_m(y),\\ &V + y + L + H_m(y),\ V + y + J,\\ &V + x + L + H_m(x),\ V + x + J\}.\end{aligned}$$

This function has exactly $m^2 + 2m + 2$ affine pieces. It admits $O(m)$ arithmetic preprocessing and storage, $O(\log m)$ value queries, and $O(m)$ complete tree construction per query.

Proof. Any root other than keys $0, m, 2m$ gives both added endpoint weights depth at least two and the central key comparison length at least two. A center-root candidate with the two outer keys as children also puts both endpoints at depth two, with base cost at most $R + n(W - R) < 2R$. It therefore strictly beats such a root for all x, y. If root 0 is used, its remaining one-sided problem similarly has only the center or the other outer key as a possible root; symmetrically for root $2m$. Expanding these five cases gives the formula. The fixed remaining intervals are geometric chains on either side of a forced central root, giving the stated L, J.

The center-root term supplies m^2 slopes, all exposed by the original probes. For root 0, the same $2R$ argument forces the center in the right problem throughout the original probe range, exposing all m pieces inherited from $H_m(y)$. As y grows unbounded, one final piece appears when the outer key becomes its root. Taking x sufficiently large forces root 0, exposing these $m+1$ pieces globally. Their slopes are $(1, 2), \ldots, (1, m+2)$. Reflection gives another $m+1$ pieces with second slope one. All slopes are distinct. The formula has no further candidate slopes, proving the count.

Prefix sums of the ordered thresholds let two binary searches evaluate $H_m(x)$ and $H_m(y)$. Four comparisons select the outer minimum. The selected case and one-sided chain indices reconstruct exactly $2m + 1$ keys in linear work. □

Thus $2(m-1)$ hinge terms and five alternatives represent a quadratic number of affine regions. Endpoint-dominance shortcuts already solve this family. It is not a hard-query example, and the facet theorem is not an expression-size lower bound. A lossless factorization also does not repair Theorem 24: the value function itself omits information needed by some surrounding contexts.

18.3 A favorable bound independent of the added weights

Lemma 28 (Endpoint mass-ratio bound)**.** *Let the original total mass be W, with $q_0, q_n > 0$. The entire response for arbitrary $x, y \geq 0$ has*

$$O(\log(2 + W/q_0) \log(2 + W/q_n))$$

affine pieces, also bounded by $O(n^2)$. Polynomial endpoint mass ratios therefore give $O(\log^2 n)$ pieces.

Proof. If an optimum has left endpoint depth $a \geq 2$, its last internal ancestor is at depth $a - 2$ in the root's left subtree. This subtree is optimal and excludes the opposite endpoint and its added weight y. Its mass is at most $W + x$, while the last ancestor's subtree has mass at least $q_0 + x$. Lemma 8 gives

$$F_{a-1} \leq \frac{W + x}{q_0 + x} \leq \frac{W}{q_0}, \qquad F_1 = F_2 = 1.$$

The symmetric bound is $F_{b-1} \leq W/q_n$ for right depth $b \geq 2$. Fibonacci growth bounds the number of possible slopes in each direction; only a least intercept per pair can be required. □

The added parameters do not occur in this bound. The quadratic example has exponential original mass ratio, so the two results are consistent. This lemma bounds output size, not the work to discover the pieces. Moreover, the composition counterexample has just five pieces: a small response need not be a sufficient interface.

Independent checks and remaining question. The exact-arithmetic harness checks 875 response queries against cubic DP on 175 inputs through seven keys. Rational half-plane clipping checks exposed pieces; 725 retained planes with positive endpoint weights satisfy the Fibonacci bound. The geometric family passes 408 hinge queries, including thresholds and neighboring integers; 1,496 two-parameter probes through 33 keys; 204 full cubic comparisons; 30 all-shape checks that every optimum has the prescribed depth pair; and 680 strict endpoint inequalities. The product checks execute 887,808 Knuth root candidates. Another 304 queries expose all outer-root pieces, and 384 queries test arbitrary parameters and the global transition threshold. All 2,184 factored complete constructions agree with Knuth and direct evaluation, and construct exactly n keys. Full-quadrant envelopes through nine keys have 5, 10, 17, and 26 pieces, agreeing with the proved formula. The factored constructions use 14,492 threshold comparisons, 8,736 outer-minimum comparisons, and construct 52,232 keys in total; these categories exclude the independently executed reference solvers.

The collision search found an initial surrounding context with outer weights 16; reducing them to seven preserves the one-unit optimum difference. The published certificate independently checks domination of discarded planes and enumerates both full eight-key tree sets. It does not depend on the clipping implementation or sampled parameters. Code, raw results, and the harness source hash are in `boundary_response.py/json`.

A richer representation must preserve some information about boundary path decisions or internal alternatives discarded by the isolated value response. The present results rule out two specific shortcuts; they do not resolve or replace the full mixed-weight subquadratic objective.

19 A core-sparse Monge reduction with terminal penalties

The preceding interface counterexample motivates preserving the actual interval costs in compressed form. We now give a reduction to the published core-sparse Monge product of Gawrychowski, Gorbachev, and Kociumaka [10]. It yields a smaller exponent, up to logarithms, for linear total integer mass than the frontier bound above, and permits zero weights. Dense reference audits and condensed Python/C++ constructors check the reduction. The compact backend is slower than matched Knuth construction at tested sizes; priority of this application is unestablished.

Let $n \geq 1$, let all key and gap weights be nonnegative integers, and put $W = \sum p_i + \sum q_i$ and $L = \lceil \log_2(n+1) \rceil$. Write $f(W)$ for the first index with $F_{f(W)} > W$, where $F_0 = 0, F_1 = 1$. Set

$$H = \min\{n, f(W) + L\}, \qquad M = (H+1)W + 1.$$

If every key weight is positive, $H = \min\{n, f(W)\}$ suffices.

Theorem 29 (Integer-mass sparse-product reduction)**.** *Using the core-sparse Monge multiplication and witness algorithms of [10], a full optimal BST can be constructed in*

$$O\Big(H\left[n + \min\{n^2, n\sqrt{WL}\}\right]\log(n+1)\Big)$$

arithmetic operations. Retaining the witness structures uses $O(H[n + \min\{n^2, n\sqrt{WL}\}])$ words. In particular, $W = O(n)$ gives $O(n^{3/2}\log^{5/2}(n+1))$ exact construction. Intermediate integers have $O(\log(n+1) + \log(W+1))$ bits.

The dependence on numerical mass is essential to this statement. The linear-mass case fits ordinary $O(\log n)$-bit arithmetic, but arbitrary large integer weights do not give an unrestricted strongly subquadratic bound. Knuth remains a fallback when the displayed bound is unfavorable.

19.1 An upper iteration that finishes in a bounded number of levels

Initialize $A_0(i,j) = M(j-i)$ for $i \leq j$, and iterate

$$\begin{aligned} A_h(i,i) &= 0, \\ A_h(i,j) &= W(i,j) + \min_{i \leq r < j}\{A_{h-1}(i,r) + A_{h-1}(r+1,j)\} \quad (i<j). \end{aligned}$$

This is an h-level partial search tree: explicit decisions incur the usual interval mass, while each key still in an unfinished terminal interval incurs penalty M. If t keys are explicit, the cost is

$$M((j-i)-t) + D, \qquad 0 \leq D \leq hW(i,j).$$

At most $\min\{j-i, 2^h-1\}$ keys can be explicit, and that number is attainable with inorder labels. Since $M > HW$, every minimizer for $h \leq H$ uses that maximum number. Writing $c_h = 2^h - 1$ gives

$$A_h(i,j) = M\max\{0, j-i-c_h\} + R_h(i,j), \qquad 0 \leq R_h(i,j) \leq hW(i,j). \tag{11}$$

The residual R_h is not asserted to be Monge.

The initial bound dominates every interval optimum: any complete tree on $j-i$ keys costs at most $(j-i)W(i,j) \leq M(j-i)$. The recurrence is monotone, and $A_1 \leq A_0$ because $W(i,j) \leq M$. Thus the iterates decrease and remain at least C. Whenever an optimal full tree fits within h levels, it is feasible without a terminal penalty, so $A_h = C$.

The mixed Fibonacci mass lemma applies here to an *actual optimal full tree*. Every positive-mass internal subtree at depth d satisfies $F_{d+1} \le W$. Zero-mass subtrees can be replaced by balanced trees of height at most L without changing cost. Therefore some optimum has height at most $f(W) + L$; the trivial bound n also applies. When all keys have positive integer weight every internal subtree has mass at least one, so the extra L is unnecessary. These arguments apply to every interval, proving $A_H = C$ throughout the upper triangle.

An earlier proposed iteration started from the additive singleton lower bound. Its attempted rotation proof of logarithmic convergence was invalid: a rotation can lift an unfinished interval, requiring extra refinement to restore the prescribed number of levels. Small passing tests do not remove that issue. The upper-penalty proof above avoids that argument.

19.2 Finite Monge products, including the diagonal boundary

For a triangular matrix A with zero diagonal, define its interior density by

$$\Delta_A(i,j) = A(i,j+1) + A(i+1,j) - A(i,j) - A(i+1,j+1) \quad (0 \le i < j < n).$$

A *core* lists nonzero densities. To use a finite matrix product, put $P = 4nM + 4W + 1$ and complete each layer by

$$E_h(i,j) = \begin{cases} A_h(i,j), & i \le j, \\ P(i-j), & i > j. \end{cases}$$

We prove by induction that E_h is Monge. This is immediate for E_0. Given the previous completion, multiply its first n columns by its last n rows in the min-plus sense:

$$B_h(i,j) = \min_{0 \le r < n} \{E_{h-1}(i,r) + E_{h-1}(r+1,j)\}.$$

For $i < j$, every invalid root has a term at least P, whereas a valid pair costs at most nM. Thus the finite product gives the restricted root minimum exactly. The product is Monge. Adding $W(i,j)$, a row term plus a column term, leaves density unchanged in squares entirely above the diagonal. The only interior squares affected by resetting a diagonal entry to zero have $j = i + 1$. Their new density is

$$\min\{A_{h-1}(i,i+1), A_{h-1}(i+1,i+2)\} - q_{i+1} \ge 0.$$

For the first step both old singleton entries are M; thereafter each is its singleton interval mass. The completion adds exactly the n diagonal densities $P + A_h(i,i+1)$; all densities below them vanish. This proves Monge preservation, including the boundary cases. All entries are integers.

19.3 A bound on every intermediate core

Let K_h be the number of nonzero interior densities of A_h. A direct telescoping identity is

$$\sum_{i<j} \Delta_A(i,j) = A(0,n) - \sum_{i=0}^{n-1} A(i,i+1). \tag{12}$$

For $h \ge L$, a balanced full tree is feasible, so $A_h(0,n) \le LW$. Nonnegative integrality then gives $K_h \le LW$. This final-stage argument alone would not bound the early layers, whose terminal penalties are large.

For $1 \le h < L$, the penalty term in (11) has density only on the line $j - i = c_h$. Account for that line and the main diagonal separately, using $O(n)$ cells. Tile the remaining density-index

triangle by maximal dyadic squares that do not cross either line. A power-of-two enclosing square, also split at the outer domain boundary, handles arbitrary n. At side length s there are $O(n/s)$ tiles: each parent of a maximal tile intersects one of a constant number of straight boundaries, and each boundary meets $O(n/s)$ grid squares at that scale.

On a tile $[a, b) \times [c, d)$ the penalty density vanishes. Its density sum telescopes to

$$R_h(a, d) + R_h(b, c) - R_h(a, c) - R_h(b, d) \le 2hW.$$

All densities in this tile are nonnegative integers. Negative residual densities can occur on the removed line and are not used. The number of nonzero entries in the tile is at most $\min\{s^2, 2hW\}$. Therefore

$$K_h = O\left(n + \sum_{s \text{ dyadic}} \frac{n}{s} \min\{s^2, 2hW\}\right) = O(n + n\sqrt{hW}),$$

capped by $O(n^2)$. Splitting the geometric sum at $s \asymp \sqrt{hW}$ introduces no extra logarithm. For $W = 0$, only the removed-line entries can remain. Combining the early and late bounds, including the n completion entries, yields

$$\delta(E_h) = O\Big(n + \min\{n^2, n\sqrt{WL}\}\Big) \qquad (0 \le h \le H). \tag{13}$$

This controls representation size without charging the large value of M as though it represented many separate core entries.

19.4 Compact computation and full reconstruction

Theorem 1.2 of [10] multiplies finite Monge matrices from their condensed representations in time linear in their dimensions plus input core size times its logarithm. Theorem 4.7 supplies compact witness oracles in the same time. Fact 4.1 provides logarithmic-time entry queries after core-based preprocessing. These are existing algorithms, not results claimed here.

Use them on the two contiguous submatrices of E_{h-1}. To obtain the next completion, retain the product core strictly above the first superdiagonal, where adding interval mass has zero density. Recompute the $O(n)$ diagonal and first-superdiagonal core entries, discard lower entries, and recover the top row and left column. Filtering and $O(n)$ logarithmic entry queries take $O((n+\delta)\log(n+1))$ work, including coordinate sorting if needed. The product's temporary core is also bounded linearly by the two input cores, by Theorem 1.1 of [10]. No step first enumerates a dense interval table.

Retain the product witness oracle for every layer. Starting from $(0, n, H)$, retrieve a root and recurse on its two intervals at layer $H - 1$. There are n nonempty calls. An unfinished nonempty interval at layer zero would incur at least M, but the optimum is at most $LW < M$, since $H \ge L$. Thus none survives; when $H = n$, completeness also follows from depth alone. Witness recovery costs an additional $O(n \log(n + 1))$. Equation (13), summed over H layers, proves Theorem 29, including dimensions, representation conversion, storage, and reconstruction. Since $H \le n$ and the penalties are polynomial in n, W, the stated bit-length bound follows as well.

Reference checks and implementation boundary. The dense audit `penalty_layers.py` passed 1,092 complete solves: 680 exhaustive binary-weight inputs through four keys, 384 random inputs through 32 keys, and 28 size/family fixtures through 256 keys. These include zeros, near-uniform, alternating, varying mixed, and geometric weights. All complete values and trees match Knuth and direct cost evaluation. There are 7,972 independent cubic interval checks, 8,048 all-root cell checks, 792 explicit padded-product checks, 792 all-shape truncated-layer checks, and 139,992

verified dyadic tiles. The geometric family correctly needs all 64 layers at 64 keys; no logarithmic-in-n claim is made for its exponentially large mass.

At 256 keys, the maximum interior core over all layers was 3,857 for mixed weights in $\{1, 2\}$, 6,196 for weights in $\{100, 101, 102\}$, and 5,008 for mixed weights in $\{1, \ldots, 16\}$, against 32,640 possible interior positions. These are representation counts, not runtime speedups. The audit actually computed 6,318,862 dense states and examined 12,381,033 root candidates across its full suite. It constructed 11,912 keys. Source-hashed raw data is in `penalty_layers.json`.

19.5 Implemented condensed constructors

The compact path is now implemented in Python and independently in C++. The product algorithm is the published compression, balanced core split, staircase merge, and decompression method of [10]. We credit that primitive separately from the OBST reduction. A matrix stores its top row, left column, and positive density triples. Two linear bucket passes canonicalize the triples. Boundary extraction uses one-dimensional sweeps; the product merge uses adjacent-entry recomputation along its staircase. Each relevant core row or column is scanned only a constant number of times per merge. None of these operations enumerates the full interval table.

Exact early stopping. The constructors compare consecutive completed layers by their top rows and canonical core triples; the left column is fixed. Equality is therefore equality of the whole table. A fixed point satisfies the ordinary OBST recurrence with zero diagonal. Induction on interval length proves that this recurrence has a unique solution, namely C. Stopping at such a fixed point is exact. Equality of the single full-interval cost is not used. Retained layer witnesses reconstruct a complete optimum: any unfinished nonempty terminal would contribute at least $M > HW$, exceeding the cost of an optimal tree of height at most H. All charges are nonnegative. Reconstruction uses an explicit stack and visits every original key.

Implementation costs and arithmetic. Both implementations retain ordinary integer coordinate maps and recursive witness nodes. Their witness storage is bounded by $O(H(n+K)\log(n+1))$ words, where $K = O(n + \min\{n^2, n\sqrt{WL}\})$ bounds the completed cores. This can exceed the succinct storage bound in Theorem 29 by a logarithmic factor. Compression bounds recursive dimensions by core size, and the core count halves along a recursive branch, so the time bound is preserved. No succinct rank/select implementation is claimed.

The C++ variant stops product recursion at leaves with at most 64 output cells. These bounded dense leaves cost at most a constant times the inner dimension and core size, preserving the product bound. They do not form a dense OBST interval table. The Python variant uses scalar leaves and arbitrary-precision integers. The C++ driver uses signed 128-bit integers, limits n to 8192, and rejects inputs outside the conservative domain

$$W \le \frac{2^{120} - 1}{16n^2(H + 2) + 16}.$$

Validation precedes both timed constructors. These are prototype-driver limits, not restrictions of the theorem. The benchmark executable separately runs Knuth, which allocates a dense table.

Independent evidence. The condensed matrix harness passed 1,600 random rectangular products, checking 251,779 output values and exact leftmost witnesses by enumeration, and 1,600 arbitrary submatrix selections. Sparse distance-matrix products at dimensions 128, 512, and 2048 each

retain seven nonzero output core entries from seven entries in each input. Core visits remain 673 in these three checks, while dimension scans grow. This exercises nontrivial sparse output, not just products with empty cores.

The complete Python constructor passed 1,134 solves, including 682 exhaustive binary-weight cases through four keys (including two empty-key inputs), 838 cubic checks, and 32 fixed-budget controls that disable early stopping. On 78 layers of the small controls, all 3,367 interval values and leftmost witnesses agree with explicit all-root enumeration. Full tree audits check 15,544 reconstructed keys. Geometric and folded weights through 64 keys remain adversarial controls; the geometric input requires all 64 layers.

The C++ address/undefined-behavior sanitizer run passed 1,278 complete solves, including 826 independent cubic checks, 12 Python compact comparisons, random weights through 48 keys, geometric/folded inputs through 64 keys, and all-zero and single-active-gap controls. It checked 18,248 reconstructed keys. The release run passed 1,362 solves and 88,904 reconstructed keys, including 1,302 independent Python Knuth comparisons. All release cases also compare against the separately implemented C++ Knuth constructor. The independent parent-array audit verifies inorder validity, unique key coverage, and direct mixed weighted cost. Three malformed/out-of-domain inputs are explicitly rejected by each binary. No sanitizer error was reported. Results include source hashes, compiler flags, and seeds.

Complete work and negative performance result. The release measurements use Apple Clang 21, C++20, `-O3 -DNDEBUG`, on the sponsor's M4 host. Each size/family has three trials. Both constructors include preprocessing and full tree reconstruction, use the same signed 128-bit arithmetic, and alternate execution order. Independent Python audits run outside those timings. At $n = 2048$, the following are medians; the last column is the median of the three paired compact/Knuth time ratios, which need not equal the ratio of the two median times.

Input	Max. core	Visits (10^6)	Compact (s)	Knuth (ms)	Ratio
Uniform	20,493	69.55	1.455	34.98	44.3
Mixed 1–2	54,860	164.64	3.236	34.41	92.4
Mixed 100–102	116,104	306.94	5.142	27.79	185.1
Mixed 1–16	78,225	260.37	4.694	32.96	145.2
Sparse 0–1	17,894	55.75	1.417	21.88	64.8
Alternating	30,229	102.42	1.959	34.35	56.2

Knuth examines approximately 4.19 million root candidates and fills 2,098,176 nonempty interval cells per trial at this size. A core visit and a root-candidate evaluation are different units; neither is a complete machine-instruction count. The compact counters also record core records constructed, dimension/map slots, adjacent-core scans, tiny-leaf candidate evaluations, and witness traversal. The much larger core-traversal counts expose substantial work hidden by simply reporting the maximum core size. Retained ordinary witness arrays contain between 3.68 and 9.33 million integer slots across these median fixtures. These counts are not peak RSS.

The compact backend loses every one of the 90 scaling trials at sizes 128 through 2048. At 2048 keys the median paired losses range from 44.3 to 185.1 times. Small cores establish a representation advantage; they do not establish an implementation speedup. The current constructor is a verified implementation of the reduction, not a practical replacement for Knuth at the tested sizes. Repeated core traversal, allocation, and witness-map storage are concrete remaining costs. Broad superiority and mathematical priority remain unestablished, and the unrestricted numerical-weight problem remains open.

20 Status and reproducibility

The general subquadratic question remains open. This checkpoint provides exact contraction, bounded-excess and split-height constructions, variable interval frontiers, dominant-gap separators, joint heavy-outcome cores, residual exclusion and coupled two-gap skeleton certificates, global certificates, scoped obstructions to depth and endpoint-response representations, and a core-sparse product reduction for integer-mass inputs with verified condensed constructors and measured implementation regressions. Mathematical novelty, broad practical benefit, and comparisons against other optimized prior implementations remain to be assessed.

Standard-library Python and C++ code, seeded exact results, and matched timing observations are under `obst/code` and `obst/results`. The README gives reproduction commands. The active objective remains to avoid most of the quadratic table on a substantially broader class, or establish a rigorously scoped complexity lower bound.

References

[1] D. E. Knuth. Optimum binary search trees. *Acta Informatica* 1, 14–25, 1971. doi:10.1007/BF00264289.

[2] L. L. Larmore. A subquadratic algorithm for constructing approximately optimal binary search trees. *Journal of Algorithms* 8(4), 579–591, 1987. doi:10.1016/0196-6774(87)90052-6.

[3] M. Karpinski, L. L. Larmore, and W. Rytter. Sequential and parallel subquadratic work algorithms for constructing approximately optimal binary search trees. *Proceedings of SODA*, 36–41, 1996. Author-hosted manuscript.

[4] M. Karpinski, L. L. Larmore, and W. Rytter. Same title as [3], longer preprint version, Section 5: Short height limited trees. Archived preprint text.

[5] D. S. Hirschberg, L. L. Larmore, and M. Molodowitch. Subtree weight ratios for optimal binary search trees. Technical Report 86-02, University of California, Irvine, 1986. Author-hosted report.

[6] P. Becker. Optimal binary search trees with near minimal height. 2010. arXiv:1011.1337.

[7] D. C. McDonald. On-line vertex ranking of trees. *SIAM Journal on Discrete Mathematics* 29(1), 145–156, 2015. arXiv:1401.2669.

[8] P. Bose, V. Dujmović, M. Javarsineh, and P. Morin. Asymptotically optimal vertex ranking of planar graphs. 2022 version. arXiv:2007.06455.

[9] D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE* 40(9), 1098–1101, 1952. doi:10.1109/JRPROC.1952.273898.

[10] P. Gawrychowski, E. Gorbachev, and T. Kociumaka. Core-sparse Monge matrix multiplication: improved algorithm and applications. *ESA 2025*, LIPIcs 351, 74:1–74:17. doi:10.4230/LIPIcs.ESA.2025.74.